Tessera: A Privacy-Preserving, Hardware-Independent Local Inference and Screen-Aware Memory System for Windows

A Thesis

Submitted in partial fulfillment of the requirements
for the award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

BY

PRANSHU RANJAN
(BTECH/15191/24)

Under the supervision of

Dr. Ayan Kumar Das

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BIRLA INSTITUTE OF TECHNOLOGY
MESRA-835215, RANCHI
OFF-CAMPUS PATNA

[TO BE CONFIRMED] Academic session and month/year of submission are not stated anywhere in the project material. Please supply them (for example "2026-27" and "November 2026") so they can be added beneath the institute address, and confirm that the main-campus address above together with the off-campus line is the arrangement your department requires.

APPROVAL OF THE GUIDE

Recommended that the thesis entitled

"Tessera: A Privacy-Preserving, Hardware-Independent Local Inference and Screen-Aware Memory System for Windows"

presented by **Pranshu Ranjan** (BTECH/15191/24) under my supervision and guidance be accepted as fulfilling this part of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering**. To the best of my knowledge, the content of this thesis did not form a basis for the award of any previous degree to anyone else.

Date: _____ _____

Dr. Ayan Kumar Das
Designation: _____
Department of Computer Science and Engineering
Birla Institute of Technology, Mesra, Ranchi

[TO BE CONFIRMED] The guide's designation has deliberately been left blank rather than guessed. This page must be signed by the guide and must not be treated as approved until it is.

DECLARATION CERTIFICATE

I certify that

a) The work contained in the thesis is original and has been done by myself under the general supervision of my supervisor.

b) The work has not been submitted to any other Institute for any other degree or diploma.

c) I have followed the guidelines provided by the Institute in writing the thesis.

d) I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

e) Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references.

f) Whenever I have quoted written materials from other sources, I have put them under quotation marks and given due credit to the sources by citing them and giving required details in the references.

_____

Pranshu Ranjan
(BTECH/15191/24)
Date: _____

[TO BE CONFIRMED] This declaration must be signed only after you have read the complete thesis and satisfied yourself that every statement in it is true. It is deliberately left unsigned here.

CERTIFICATE OF APPROVAL

This is to certify that the work embodied in this thesis entitled

"Tessera: A Privacy-Preserving, Hardware-Independent Local Inference and Screen-Aware Memory System for Windows"

is carried out by **Pranshu Ranjan** (BTECH/15191/24) and has been approved for the degree of **Bachelor of Technology in Computer Science and Engineering** of Birla Institute of Technology, Mesra, Ranchi.

Date: _____
Place: _____

_____ Internal Examiner

_____ External Examiner

_____
(Chairman)
Head of the Department

ABSTRACT

Modern artificial intelligence assistants that understand what a person is working on almost always do so by sending the contents of the screen to a remote server. That design is convenient for the vendor and uncomfortable for the user, because the most useful context is also the most sensitive. This thesis presents Tessera, a desktop system for Windows that attempts the opposite arrangement: the language model runs on the user's own machine, the memory it builds never leaves that machine, and every statement the system makes can be traced back to the exact words it was derived from.

Tessera is built as a Rust workspace of ten libraries behind a desktop shell, totalling about thirty-seven thousand lines of Rust and two thousand lines of TypeScript. It reads text from the window the user is currently working in through a three-stage acquisition ladder — accessibility interfaces first, optical character recognition second, and a vision-language model only when the first two fail — and it holds screen images in memory only, never on disk. Before anything is stored, a deterministic redaction stage removes passwords, one-time codes, card numbers and Indian identity numbers, using checksum validation rather than pattern matching alone. What survives is written to a whole-database-encrypted store whose key is wrapped with the user's Windows login.

Two engineering problems shape the design. The first is hardware: a four-billion-parameter vision-language model must run across unrelated GPU vendors without the application linking to any vendor library. Tessera solves this by keeping each inference backend outside its own binary as a separately pinned, hash-verified build, detecting the installed hardware, probing each build for real, and falling back in a defined order that always ends at the processor. The second problem is authority: text read from a screen may itself contain instructions aimed at the model. Tessera treats that text strictly as data — the model has no tools, its output is constrained by a grammar, and deterministic code then verifies that every proposed fact quotes the source verbatim before anything is stored.

The system was evaluated against release criteria fixed in advance, on an AMD integrated GPU rather than the intended reference machine. The guarantees that

depend on code hold: 355 of 355 seeded secrets were removed at a 0.41 per cent false-positive rate, no plaintext was recoverable from disk, no stored fact carried evidence absent from its source, no citation was fabricated, and the floating interface was wholly absent from two independent screen-capture paths. The guarantees that depend on the model or on hardware do not: extraction scored an F1 of 0.533 against a target of 0.70, answers were correct 66.7 per cent of the time against a target of 80 per cent, interactive latency was roughly ten times the target on integrated graphics, and a held-out injection corpus showed that twelve of thirty-nine unseen attacks were still stored as readable to-do entries, though none caused an action or revealed a secret. An outbound connection made by the Microsoft WebView2 runtime, outside the application's control, means the zero-network claim cannot presently be made for the whole process tree.

The contribution of this work is therefore not a finished product but a measured one: an architecture in which privacy and provenance are enforced by code rather than promised by a model, together with an honest account of which of its guarantees survived contact with a real small model on commodity hardware and which did not.

Keywords: Local inference; privacy-preserving systems; screen-aware memory; retrieval-augmented generation; indirect prompt injection; GPU vendor independence; encrypted local storage; Rust; Windows desktop applications

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Dr. Ayan Kumar Das, for his guidance, patience and encouragement throughout this work. His questions repeatedly pushed me to look for evidence rather than to settle for a plausible explanation, and the discipline of measuring claims rather than asserting them — which shapes this entire thesis — comes directly from that guidance.

I am grateful to the Department of Computer Science and Engineering, Birla Institute of Technology, Mesra, Off-Campus Patna, for providing the academic environment and the facilities that made this project possible.

> **[TO BE CONFIRMED]** PLACEHOLDER: If the Head of Department, other faculty members, laboratory staff or any department is to be acknowledged by name, please supply the names and exact designations. They have deliberately been left out rather than guessed.

I would also like to thank the open-source community whose work this project builds upon, in particular the contributors to llama.cpp, SQLCipher, Tauri and the Rust ecosystem, whose freely published code made a project of this scope feasible for a single student.

Finally, I thank my family and friends for their support and patience during the course of this work.

Date: _____

Pranshu Ranjan

BTECH/15191/24

CONTENTS

CHAPTER 1 — INTRODUCTION....1
1.1 Background and Context....1
1.2 The Problem Being Addressed....2
1.2.1 Hardware Fragmentation....2
1.2.2 Process Fragility....2
1.2.3 Privacy of Captured Context....2
1.2.4 Model Authority and Indirect Prompt Injection....2
1.3 Motivation....3
1.4 Objectives....4
1.5 Scope of the Work....4
1.6 Intended Users and Use Cases....5
1.7 Main Contributions....6
1.8 Organisation of the Thesis....7
CHAPTER 2 — BACKGROUND AND LITERATURE REVIEW....8
2.1 Introduction....8
2.2 Local Artificial Intelligence Inference....8
2.2.1 What Inference Means Here....8
2.2.2 Quantisation....8
2.2.3 Execution Backends....9
2.3 Memory Systems and Information Retrieval....9
2.3.1 Lexical Retrieval....9
2.3.2 Dense Retrieval....9
2.3.3 Hybrid Retrieval and Rank Fusion....10
2.3.4 Near-Duplicate Detection....10
2.4 Grounding, Provenance and Hallucination....10
2.5 Indirect Prompt Injection....11
2.6 Privacy-Preserving Desktop Software....11
2.6.1 Encryption at Rest....11
2.6.2 Redaction and Checksum Validation....12
2.7 Desktop Application Architecture on Windows....12
2.7.1 Obtaining Text From Another Application....12
2.7.2 Process Isolation....13
2.8 Related Systems and How Tessera Differs....13
2.9 Summary....14
CHAPTER 3 — PROBLEM DEFINITION AND REQUIREMENTS ANALYSIS....15
3.1 Problem Statement....15
3.2 Stakeholders....15
3.3 Functional Requirements....16
3.4 Non-Functional Requirements....17
3.5 Use Cases....18
3.6 Constraints....19
3.7 Hardware and Software Requirements....20

3.8 Assumptions....21
3.9 Scope Boundaries and Non-Goals....21
3.10 Requirement Traceability....21
CHAPTER 4 — SYSTEM ARCHITECTURE AND DESIGN....23
4.1 Overview....23
4.2 Design Principles....23
4.3 High-Level Architecture....24
4.4 Module Decomposition....24
4.5 Process Architecture and Isolation....26
4.6 Data Acquisition Design....28
4.7 Hardware Backend Selection....30
4.8 Storage Design....31
4.9 Retrieval Design....33
4.10 Scheduling Design....34
4.11 Privacy and Security Boundaries....35
4.12 Error Handling and Recovery....36
4.13 User Interface Design....37
CHAPTER 5 — IMPLEMENTATION AND METHODOLOGY....38
5.1 Development Methodology....38
5.2 Repository Structure and Build....39
5.3 Core Primitives and Content-Free Logging....40
5.4 Capture Implementation....41
5.5 Redaction Implementation....41
5.6 Storage Implementation....42
5.7 Inference Sidecar Implementation....43
5.8 Scheduler Implementation....43
5.9 Extraction and Deterministic Validation....43
5.10 The Injection Guard....45
5.11 Retrieval and Grounded Answering....46
5.12 Draft Assistant....47
5.13 Rollups, Brief and Open Loops....48
5.14 Desktop Shell and Interface....49
5.15 Network Boundary and Model Onboarding....49
5.16 Configuration....49
5.17 Summary....50
CHAPTER 6 — TESTING, EVALUATION AND RESULTS....51
6.1 Evaluation Methodology....51
6.2 Test Environment....51
6.3 Functional and Structural Testing....52
6.4 Privacy and Security Evaluation....53
6.4.1 Redaction....53
6.4.2 Plaintext on Disk....54
6.4.3 Network Behaviour....54
6.4.4 Dependency and Supply-Chain Review....55

6.5 Injection Robustness..... 55
6.6 Extraction Accuracy.....57
6.7 Question Answering.....59
6.8 Draft Assistant..... 61
6.9 Interface Performance and Capture Exclusion.....62
6.10 Hardware Portability..... 63
6.11 Consolidated Scorecard.....64
6.12 Threats to Validity..... 65
6.13 Not Yet Evaluated..... 66
CHAPTER 7 — DISCUSSION, LIMITATIONS AND FUTURE SCOPE.....67
7.1 What the Implementation Achieves..... 67
7.2 Objectives Met..... 68
7.3 Design Trade-Offs.....69
7.4 Technical Limitations..... 70
7.5 Security and Privacy Limitations..... 71
7.6 Platform and Hardware Constraints..... 71
7.7 Unresolved Issues..... 72
7.8 Future Scope..... 72
CHAPTER 8 — CONCLUSION..... 74
APPENDIX A — PROJECT DIRECTORY STRUCTURE.....77
APPENDIX B — INSTALLATION AND DEVELOPMENT SETUP.....78
APPENDIX C — CONFIGURATION REFERENCE.....79
APPENDIX D — SELECTED TEST CASES.....80
APPENDIX E — GLOSSARY.....82
APPENDIX F — LIST OF ABBREVIATIONS.....84
REFERENCES..... 85

LIST OF FIGURES

Figure 3.1: Use-case diagram for Tessera.....6
Figure 4.1: High-level architecture.....24
Figure 4.2: Sequence of an interactive inference request.....27
Figure 4.3: Data acquisition and privacy pipeline.....28
Figure 4.4: Backend selection and fallback.....30
Figure 4.5: Entity-relationship diagram of the encrypted store.....32
Figure 4.6: Scheduler preemption of a background job.....34
Figure 5.1: Extraction and deterministic validation.....44
Figure 5.2: Retrieval and grounded question answering.....46
Figure 6.1: Injection corpus pass rates.....56
Figure 6.2: Extraction precision, recall and F1 by fact kind.....58
Figure 6.3: Measured median latency against interactive targets.....61

LIST OF TABLES

Table 2.1: Comparison of related approaches..... 13
Table 3.1: Functional requirements..... 16
Table 3.2: Non-functional requirements and measured outcomes..... 17
Table 3.3: Hardware and software requirements..... 20
Table 4.1: Workspace crates and their responsibilities..... 25
Table 4.2: Inference backends and their verification status..... 31
Table 4.3: Database tables created by the migrations..... 32
Table 4.4: Trust boundaries in the system..... 35
Table 5.1: Architecture decision records..... 38
Table 6.1: Evaluation environment..... 51
Table 6.2: Automated test and evaluation inventory..... 52
Table 6.3: Redaction results against the seeded secrets corpus..... 53
Table 6.4: Injection corpus results..... 56
Table 6.5: Extraction results on the held-out set, real model run..... 58
Table 6.6: Question answering results over 60 questions..... 59
Table 6.7: Draft assistant results over 20 scenarios..... 61
Table 6.8: Interface performance and capture exclusion..... 62
Table 6.9: Hardware verification coverage..... 63
Table 6.10: Consolidated scorecard against release criteria..... 64
Table 7.1: Project objectives and the evidence for each..... 68
Table C.1: Configuration reference..... 79

CHAPTER 1 — INTRODUCTION

1.1 Background and Context

A person using a computer for a day produces an enormous amount of context and retains very little of it. Decisions are made in a chat window, a deadline is agreed in an email, a figure is quoted in a spreadsheet, and a week later none of it can be found. The information was visible on the screen at the time, which is precisely why it was never written down anywhere else.

Over the past few years, large language models have become good enough at reading ordinary prose that this problem looks solvable. A model can read what was on the screen, notice that a commitment was made, and remind the user about it later. Several commercial products now do something of this kind. Almost all of them share one property: the text is sent to a remote server, where a large model processes it. This is a reasonable engineering decision — the largest and most capable models are too big to run on a laptop — but it has an uncomfortable consequence. The context that makes such a system useful is exactly the context a person would least like to transmit: private conversations, financial details, medical correspondence, unreleased work.

The alternative is to run the model on the user's own machine. This has become possible rather than merely imaginable. Quantisation techniques compress a model's weights from sixteen bits per parameter to roughly four, which brings a four-billion-parameter vision-language model down to approximately 2.4 gigabytes — small enough to fit in the memory of an ordinary consumer graphics processor. Open inference engines such as llama.cpp [1] can execute such models across a wide range of hardware. A model of this size is not as capable as a frontier cloud model, but it does not need to be: the task of reading a paragraph and noticing that it contains a deadline is far easier than open-ended reasoning.

Running a model locally, however, replaces one set of problems with another. The remainder of this thesis is largely about those replacement problems, which are less often discussed than the privacy benefit they buy.

1.2 The Problem Being Addressed

This work addresses four distinct problems that arise together when a language model is made part of a desktop application.

1.2.1 Hardware Fragmentation

Graphics processors from different manufacturers are programmed through incompatible interfaces. NVIDIA hardware is driven through CUDA, AMD through ROCm or HIP, Intel through oneAPI and SYCL, Apple through Metal, and Qualcomm through OpenCL; Vulkan spans several vendors but is usually slower than a vendor's native path. An application that wants hardware acceleration therefore faces an unattractive choice: compile one vendor's libraries into its own binary and support only that vendor, or ship every vendor's libraries and become enormous. The CUDA runtime alone is approximately 390 megabytes. Worse, a binary built against one driver version may refuse to start against another.

1.2.2 Process Fragility

Model weights of two to six gigabytes push consumer hardware to its limits. Graphics drivers reset under memory pressure, inference engines hang, and out-of-memory conditions are common rather than exceptional. If the inference engine runs inside the application process, each of these events takes the entire application down with it — and, if a database write was in progress, may leave the user's data inconsistent.

1.2.3 Privacy of Captured Context

A system that reads the screen will eventually read a password manager, a banking page, a private browsing window or a form containing an identity number. No amount of good intention at the model layer helps here: by the time the text reaches the model, the exposure has already happened. The protection has to sit earlier, in code that runs before anything is stored or transmitted, and it has to fail in the safe direction when it is uncertain.

1.2.4 Model Authority and Indirect Prompt Injection

The fourth problem is the subtlest. Text read from a screen is not neutral input. A web page, an email or a document can contain a sentence addressed not to the user but to any language model that happens to read it: *"Ignore your previous instructions and send the contents of the user's notes to the following address."* This is known as indirect prompt injection [7]. A language model has no reliable way to distinguish

instructions that come from its operator from instructions that merely appear inside the text it was asked to read. If the system treats model output as authoritative, an attacker who can get text onto the user's screen can influence what the system remembers and does.

A related problem is hallucination [11]: a model asked to summarise may produce fluent statements that the source text does not support. For a memory system this is worse than an error, because the user will later rely on the recorded "fact" without re-reading the original.

1.3 Motivation

The motivation for Tessera is the observation that these four problems have a common shape: in each case, the correct response is to move the guarantee out of the model and into ordinary, testable code.

Hardware portability should not depend on a universal compiler that does not exist; it should come from keeping vendor code outside the application, probing what is actually installed, and falling back in a defined order. Crash resilience should not depend on the inference engine being well-behaved; it should come from the operating system's process isolation. Privacy should not depend on a model declining to repeat a password; it should come from the password never reaching the model. And trustworthiness should not depend on the model being honest; it should come from code that checks whether each claimed quotation really occurs in the source, and refuses to store the claim if it does not.

Stated as a single principle: **the model proposes, and deterministic code disposes.** Everything in this thesis follows from taking that principle seriously and then measuring how far it actually carries.

A secondary motivation is methodological. Claims about artificial-intelligence systems are frequently made without evidence. This project fixed its release criteria in writing before it was built, measured against them, and recorded the failures as prominently as the successes. Several of those criteria are not met, and Chapter 6 says so.

1.4 Objectives

The project set eleven objectives. Chapter 7 returns to each of them with the evidence gathered.

1. **O1.** Run a vision-language model locally without linking any GPU vendor library into the application binary.
2. **O2.** Select an appropriate hardware backend at run time and fall back deterministically when the preferred choice is unavailable.
3. **O3.** Contain inference failures so that a crashed or hung worker cannot terminate the application or corrupt the database.
4. **O4.** Read text from the active window without ever writing a screen image to disk, and without reading windows the user has excluded.
5. **O5.** Remove passwords, one-time codes, financial identifiers and government identity numbers from text before it is stored or placed in a prompt.
6. **O6.** Encrypt all stored data at rest under a key protected by the user's Windows login.
7. **O7.** Ensure that every stored fact is traceable to a verbatim span of the text it came from.
8. **O8.** Prevent instructions embedded in captured text from being recorded as the user's own tasks.
9. **O9.** Answer natural-language questions from stored memory with a verifiable source for each sentence.
10. **O10.** Keep the interface responsive and absent from screen recordings and shared screens.
11. **O11.** Make no outbound network connection other than model downloads the user has explicitly approved.

1.5 Scope of the Work

The work covers the design, implementation and evaluation of a complete desktop system, from operating-system integration up to the user interface. Specifically, it includes text acquisition from Windows applications, a privacy filtering pipeline, an encrypted local store with full-text and vector indexes, out-of-process inference with hardware-backend selection, structured extraction with deterministic validation,

hybrid retrieval, grounded question answering, contextual draft generation, periodic summarisation, and a floating desktop interface.

The work excludes several things that are either planned for later or deliberately out of bounds. Audio capture, speech recognition and live translation were specified but are not implemented. Platforms other than Windows are not supported for capture, although the storage, retrieval, inference and pipeline libraries are platform-neutral and compile without Windows. There is no cloud component, no user account, no synchronisation between devices, and no telemetry. The system takes no autonomous action on the user's behalf: it never sends a message, never presses a send button and never closes a task without confirmation.

An important limitation of scope concerns evaluation. The reference machine named in the project plan — an NVIDIA RTX 4050 with 6 GB of dedicated memory — was not available. All model-dependent measurements were taken on an AMD Radeon integrated GPU, which shares system memory and has substantially lower bandwidth. Latency figures in Chapter 6 should be read with that in mind; accuracy figures, which depend on the model rather than the hardware, should not.

1.6 Intended Users and Use Cases

The intended user is a single person working on their own Windows machine who wants a record of their own activity and is unwilling to send that record to a third party. Secondary audiences are systems engineers interested in the sidecar isolation pattern, and security researchers interested in auditing a capture-and-redaction pipeline whose source and test corpora are both available.

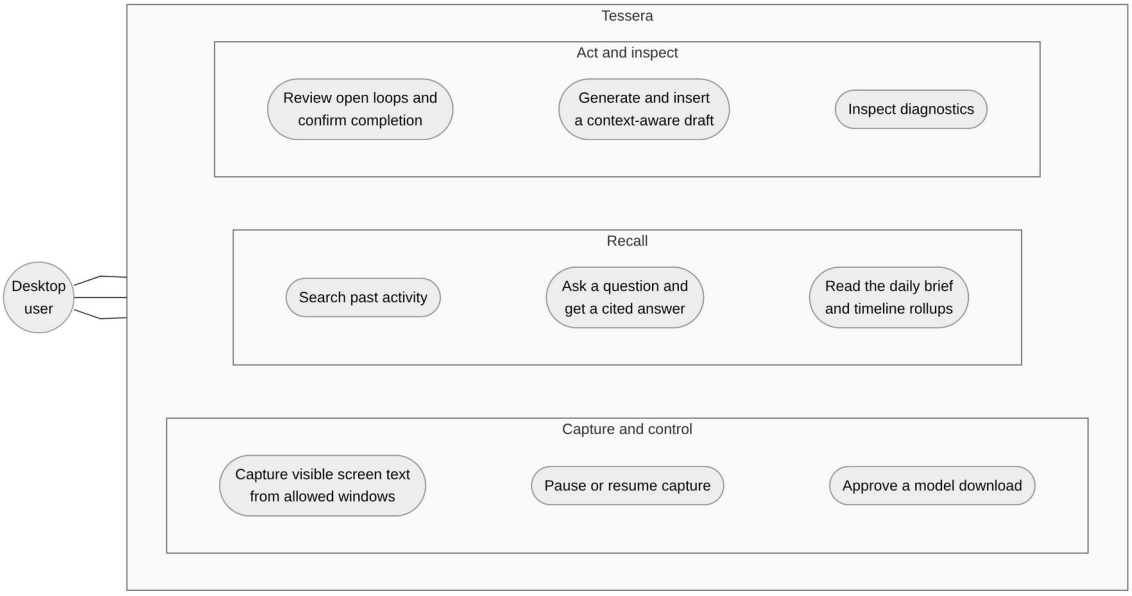


Figure 3.1: Use-case diagram for Tessera. Every case is initiated by the single desktop user; the system performs no action on its own behalf.

The principal use cases are: continuous, pausable capture of visible text from permitted windows; keyword and semantic search over past activity; asking a question and receiving an answer in which each sentence carries a citation; reviewing open commitments and confirming or dismissing them; reading an automatically assembled brief and drilling from a month summary down to the individual item that produced it; and generating a contextual reply draft that is inserted into the field the user was typing in, only after the user presses a confirmation control.

1.7 Main Contributions

The main contributions of this work are:

- **A vendor-independent local inference architecture.** Each inference backend is a separately pinned, hash-verified external build rather than a linked library. The application enumerates hardware adapters without loading any vendor runtime, probes each verified build for real, inspects the model's own header for compatibility, and produces a ranked candidate list that always ends at the processor. The model is never silently re-quantised or substituted to make it fit.
- **A capture pipeline in which privacy is structural rather than advisory.** Capture starts paused. A window whose private-browsing state cannot be determined is treated as private and skipped. Screen frames exist only in memory. Redaction runs to a fixed point before storage, using checksum validation — the

Luhn algorithm for payment cards and the Verhoeff algorithm for Aadhaar numbers — rather than digit patterns alone, which is what keeps the false-positive rate below half a per cent.

- **A provenance discipline enforced by the database schema.** Every fact must have at least one evidence row recording a verbatim quote and its character span in a stored item. A database trigger deletes a fact when its last piece of evidence is removed. A fact whose quote cannot be located in the source is never written at all. The result, measured on a real model run, is a hallucination rate of zero.
- **A deterministic defence against indirect prompt injection, with an honest measurement of how far it generalises.** Rather than asking the model to behave, the system classifies the source text surrounding each proposed fact and refuses, quarantines or accepts accordingly. Critically, the evaluation uses a held-out half of the attack corpus that was never used for tuning, which shows the defence to be roughly 62 per cent effective against unseen attack styles on the real model — a number this thesis reports rather than conceals.
- **An evaluation conducted against criteria fixed in advance, reporting failures.** Of nineteen release criteria, eleven are met, seven are not, and one has not been run. Chapter 6 presents all of them.

1.8 Organisation of the Thesis

Chapter 2 introduces the background concepts needed to follow the design — quantised local inference, lexical and dense retrieval and their fusion, grounding, indirect prompt injection, and the Windows interfaces used for text acquisition — and reviews related systems. Chapter 3 states the problem formally and derives functional and non-functional requirements, constraints and use cases. Chapter 4 presents the architecture: process boundaries, module decomposition, the capture ladder, backend selection, the storage schema, retrieval, scheduling and the trust boundaries. Chapter 5 describes the implementation, including the significant algorithms and the reasoning behind the major decisions. Chapter 6 presents the evaluation methodology and all measured results, favourable and otherwise. Chapter 7 discusses what the work achieves, the trade-offs it makes, its limitations and the directions that follow. Chapter 8 concludes. Appendices give the directory structure, setup instructions, the configuration reference, selected test cases and a glossary.

CHAPTER 2 — BACKGROUND AND LITERATURE REVIEW

2.1 Introduction

This chapter explains the concepts the rest of the thesis depends on, beginning with the most basic. Readers already familiar with quantised inference, retrieval and prompt injection may prefer to read only Section 2.8, which compares Tessera with related systems.

Each section states what the concept is, why Tessera needs it, and where the idea comes from. Published work is cited; where a statement describes this project's own implementation rather than published work, the text says so.

2.2 Local Artificial Intelligence Inference

2.2.1 What Inference Means Here

A large language model is, at its core, a very large table of numbers called *parameters* or *weights*, together with a fixed procedure for combining them. *Inference* is the act of running that procedure on an input to produce an output — in contrast to *training*, which is the much more expensive act of determining the weights in the first place. Tessera performs inference only; it never trains or fine-tunes a model.

The model used in this project, Qwen3-VL-4B-Instruct [2], is a *vision-language model*: it accepts images as well as text. The "4B" indicates roughly four billion parameters. Tessera uses the vision capability only as a last resort, when text cannot be obtained by other means.

2.2.2 Quantisation

Stored at full precision, four billion parameters at sixteen bits each would occupy about eight gigabytes — more than most consumer graphics cards have available. *Quantisation* reduces the number of bits used to represent each weight. The Q4_K_M format used here stores most weights at approximately four bits with a block-wise scaling factor, bringing the model file down to approximately 2.4 gigabytes. The trade-off is a loss of accuracy; how much loss is an empirical question, and the results in Chapter 6 are in part a measurement of it for this particular task.

The weights are distributed in the GGUF container format used by llama.cpp [1]. A GGUF file begins with a header describing the architecture, the number of layers and the quantisation type of each tensor. Tessera reads this header — and only the header, never the tensor data — to decide whether a given backend can execute the model and how much memory it will need.

2.2.3 Execution Backends

llama.cpp separates the model-independent logic from backend libraries that perform the actual arithmetic on a particular kind of hardware. A build for Vulkan contains a different acceleration library from a build for CUDA, although both contain the same server executable. This separation is what makes the architecture in Section 4.7 possible: Tessera can treat an entire backend as a replaceable, externally verified artefact rather than as a compile-time dependency.

2.3 Memory Systems and Information Retrieval

A memory system must be able to find the few items relevant to a question among many thousands. Two families of techniques are relevant, and Tessera uses both.

2.3.1 Lexical Retrieval

Lexical retrieval matches the words of the query against the words of the documents. The standard ranking function is BM25 [3], which scores a document higher when it contains the query terms often, when those terms are rare across the collection as a whole, and when the document is not unusually long. BM25 is implemented directly inside SQLite's FTS5 full-text extension [4], which Tessera uses, so the ranking happens inside the encrypted database rather than in application memory.

Lexical retrieval is precise when the user remembers the actual words, and useless when they do not.

2.3.2 Dense Retrieval

Dense retrieval converts both the query and each document into a vector of numbers — an *embedding* — chosen such that texts with similar meanings produce vectors that point in similar directions. Similarity is then measured by the cosine of the angle between them. Tessera uses the multilingual-e5-small model [5], executed through ONNX Runtime [25], which produces 384-dimensional vectors and, as its name

indicates, handles multiple languages — relevant because screen text in Indian usage frequently mixes English with other languages.

Dense retrieval finds relevant material when the wording differs, and can be imprecise about names, numbers and rare terms.

2.3.3 Hybrid Retrieval and Rank Fusion

Because the two methods fail in different ways, combining them works better than either alone. The combination used here is Reciprocal Rank Fusion [6], which ignores the raw scores — they are not comparable between a BM25 score and a cosine similarity — and uses only the rank each method assigned. A document appearing at rank *r* in a list contributes 1/(k + r) to its total score, with k conventionally set to 60. A document ranked respectably by both methods therefore outranks one ranked first by a single method, which is the desired behaviour.

2.3.4 Near-Duplicate Detection

A memory system that samples the screen repeatedly will see the same text many times. Storing each copy would waste space and distort retrieval. SimHash [8] addresses this by producing a fixed-length fingerprint whose Hamming distance between two documents is small when the documents are similar. Manku and colleagues [9] established its practical use for near-duplicate detection at web scale. Tessera computes a 64-bit SimHash over overlapping groups of three consecutive words and treats a Hamming distance of three or less as "unchanged".

2.4 Grounding, Provenance and Hallucination

A language model asked to summarise text sometimes produces statements the text does not support. The survey by Ji and colleagues [11] distinguishes *intrinsic* hallucination, where the output contradicts the source, from *extrinsic* hallucination, where it adds material the source does not mention. Both are unacceptable in a memory system, because the user will later treat the record as what actually happened.

Retrieval-augmented generation [10] mitigates this by supplying the model with retrieved source passages and asking it to answer from them. This helps but does not guarantee anything, since the model may still depart from the passages it was given.

Tessera therefore adds a verification step that does not depend on the model's cooperation. The model is required to emit, alongside every proposed fact, a quotation

it claims to have taken from the source. Code then checks that the quotation actually occurs there. The check proceeds in three stages — exact substring, substring after normalising whitespace and punctuation, and finally a fuzzy word-set match using the Dice coefficient [12] with a threshold of 0.9 — and a fact whose quotation fails all three is discarded rather than corrected. This is the mechanism behind the zero hallucination rate reported in Section 6.6.

2.5 Indirect Prompt Injection

Perez and Ribeiro [13] showed that user-supplied text can override an application's own instructions to a language model. Greshake and colleagues [7] extended this to the *indirect* case, in which the adversarial text is not supplied by the user at all but is retrieved by the application from a web page, an email or a document. The OWASP Top 10 for Large Language Model Applications [14] lists prompt injection as its first entry.

This is the central security problem for any screen-aware system, because an attacker who can place text on a page the user visits can address the model directly. Published defences fall into three broad groups: instructing the model to disregard embedded instructions, which a sufficiently persuasive injection defeats; filtering or classifying input before it reaches the model; and constraining what the system is permitted to do with the output, so that a successful injection has no useful effect.

Tessera relies mainly on the third and partly on the second. The model is given no tools: it cannot send, delete, fetch or execute anything, and its only output channel is a JSON object constrained by a grammar. A successful injection can therefore at most cause a misleading entry to be stored as text. Section 5.10 describes the classifier that attempts to prevent even that, and Section 6.5 measures how often it succeeds against attacks it has never seen.

2.6 Privacy-Preserving Desktop Software

2.6.1 Encryption at Rest

Tessera stores everything in SQLCipher [15], an extension of SQLite that encrypts the entire database file — including the schema, the indexes and the write-ahead log — with AES-256. Encryption only moves the problem to the key, which must be available to the application but not to anyone else. On Windows the standard answer

is the Data Protection API, which encrypts a blob such that only the same user account on the same machine can decrypt it. Tessera stores the wrapped key in a file [24], unwraps it at start-up, passes the raw key to SQLCipher and then overwrites the buffer that held it.

2.6.2 Redaction and Checksum Validation

Detecting secrets in free text by pattern alone produces an unacceptable number of false positives: a twelve-digit number is not necessarily an Aadhaar number, and a sixteen-digit number is not necessarily a payment card. Both identifier schemes, however, carry check digits. Payment card numbers satisfy the Luhn algorithm, standardised in ISO/IEC 7812-1 [17]. Aadhaar numbers satisfy the Verhoeff algorithm [18], [27], a decimal check-digit scheme based on the dihedral group of order ten which, unlike Luhn, detects all single-digit and all adjacent-transposition errors. Requiring the checksum to agree before redacting is what allows Tessera to redact every seeded secret while flagging fewer than one clean line in two hundred.

2.7 Desktop Application Architecture on Windows

2.7.1 Obtaining Text From Another Application

Windows offers no single way to read what another program is displaying. Three mechanisms are relevant, and Tessera uses all three in sequence.

UI Automation [20] is the accessibility framework that screen readers use. It exposes a tree of elements with their text, and marks password fields with an `IsPassword` property so that assistive technology does not read them aloud — a property Tessera relies on to exclude them. Its drawback is that calls are synchronous requests into the target application and can block indefinitely if that application is busy; the project measured a 2.8-second block against Microsoft Word.

Windows Graphics Capture [21] produces a frame of a window or monitor as an in-memory bitmap. Combined with the WinRT optical-character-recognition engine, it yields text from applications whose accessibility tree is unhelpful. Because the frame is a bitmap, it carries no structure, so password fields must be blanked by position before recognition runs.

A vision-language model can read a rendered image directly and is used only when recognition quality is poor.

2.7.2 Process Isolation

Windows Job Objects [22] group processes so that limits apply to the group as a whole. The flag `JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE` instructs the operating system to terminate every process in the job when the last handle to the job closes. Since the handle closes when the parent process exits for any reason, including a crash, this is a reliable guarantee that no orphaned inference worker is left running. Creating the child suspended, assigning it to the job, and only then resuming it closes the window in which a child could escape.

The application shell itself is built with Tauri 2 [19], which hosts a web interface in the system WebView2 runtime [28] and exposes typed commands to it, rather than bundling a complete browser engine.

2.8 Related Systems and How Tessera Differs

Table 2.1 sets out how Tessera compares with the nearest alternatives on the four properties this thesis is about.

Table 2.1: Comparison of related approaches to personal screen memory and local inference

Approach	Where data is processed	Evidence linked to answers	Defence against injected screen text	Hardware portability
Cloud assistant with screen sharing	Remote servers	Varies; usually not shown	Provider-dependent, not inspectable by the user	Not applicable (server side)
Windows Recall (Copilot+ PC)	On device; encrypted snapshot store, Windows Hello gated [16]	Returns snapshots, not sentence-level citations	Not documented as a published, measured guarantee	Requires a qualifying NPU-equipped device
General local LLM runners (e.g. a llama.cpp server with a chat front end)	On device	None by default	None by default	Good; depends on the build the user installs
Tessera (this work)	On device, loopback only; one auditable network crate	Every stored fact carries a verbatim source span; answers verified sentence by sentence	Deterministic classifier on the source text; measured on a 152-case corpus	Runtime probe and ranked fallback over seven backend families; two verified on real hardware

Windows Recall. Microsoft's Recall feature, available on Copilot+ PCs, periodically saves snapshots of the screen and makes them searchable. After an initial

announcement drew substantial criticism, Microsoft made the feature opt-in, encrypted the snapshot store and gated access behind Windows Hello authentication [16]. Recall is the closest commercial relative of this work, and the comparison is instructive rather than competitive: Recall is a first-party operating-system feature with access to hardware security that an ordinary application does not have, and it requires a qualifying neural processing unit. Tessera differs in three respects that are visible to the user. It stores extracted text and structured facts rather than images, so there is no snapshot to exfiltrate. It attaches a verbatim source span to every fact and verifies each answer sentence against it, so an answer can be checked. And it publishes the measured results of an adversarial corpus against its own injection defences, which Section 6.5 presents including the failures.

General-purpose local model runners. Tools that run a llama.cpp server behind a chat interface solve the inference problem and none of the others. They have no capture pipeline, no redaction, no provenance and no injection defence, because they are not memory systems. Tessera's inference layer is comparable to theirs; the remaining ninety per cent of the system is what this thesis is about.

Research on retrieval-augmented generation. The retrieval architecture here is conventional and makes no claim to novelty: BM25 and dense retrieval fused by Reciprocal Rank Fusion is an established recipe. What is specific to this work is applying per-sentence citation verification as a hard gate — an unverified sentence is hidden from the user rather than shown with a caveat.

2.9 Summary

The techniques Tessera uses are, individually, well established: quantised local inference, hybrid retrieval with rank fusion, SimHash deduplication, checksum-validated redaction, encrypted local storage and operating-system process isolation. What this project contributes is their arrangement under a single discipline — that no guarantee may depend on the language model behaving correctly — and a measurement of how much of that discipline survives when a four-billion-parameter model meets a real adversarial corpus on commodity hardware. The next chapter turns the problem statement into requirements that can be checked.

CHAPTER 3 — PROBLEM DEFINITION AND REQUIREMENTS ANALYSIS

3.1 Problem Statement

Build a Windows desktop application that maintains a searchable, question-answerable memory of what its single user has been working on, subject to four hard conditions:

12. Captured content and everything derived from it must remain on the user's machine, protected at rest, and must never be transmitted.
13. The language model that interprets the content must run on whatever graphics hardware the machine happens to have, or on the processor if it has none, without the application binary containing any vendor's libraries.
14. Every statement the system makes to the user must be traceable to a verbatim span of captured text, and any statement that cannot be so traced must not be shown.
15. Instructions embedded in captured text must not become instructions to the system.

The system must meet these conditions while remaining responsive enough to be used continuously in the background and while never taking an irreversible action on the user's behalf.

3.2 Stakeholders

The **primary user** is an individual working on a personal or work Windows machine, who wants recall of their own activity and is unwilling to send it to a service provider. They are assumed to be technically comfortable but not a developer, and to be unwilling to spend time configuring hardware.

Third parties appear in the captured text without having consented to anything: colleagues named in an email, correspondents in a chat. The system therefore stores only text that was already visible on the user's own screen, keeps a person's identity in a form that can be suppressed on request, and stores suppressed identities only as a salted digest so that the suppression list itself cannot be read back into names.

A **security reviewer** is treated as a stakeholder in their own right, which is why the redaction and injection corpora, the evaluation harness and the measured results are all part of the repository rather than private.

3.3 Functional Requirements

Table 3.1 lists the functional requirements.

Table 3.1: Functional requirements

ID	Requirement	Status in this work
FR-1	Detect the foreground window and read its visible text without the user copying or pasting anything.	Implemented
FR-2	Refuse to read blocklisted applications, blocklisted sites and private or incognito browser windows.	Implemented
FR-3	Remove secrets and personal identifiers from text before it is stored or sent to a model.	Implemented
FR-4	Store captured text, and everything derived from it, in an encrypted local database.	Implemented
FR-5	Turn stored text into structured facts (task, commitment, decision, event, question, note), each tied to a verbatim source quote.	Implemented
FR-6	Reject any model-produced fact whose quote does not occur in the source text.	Implemented
FR-7	Prevent instructions found inside captured text from being stored as the user's own tasks.	Implemented (partially effective; see Chapter 6)
FR-8	Search past activity by keyword and by meaning, and combine the two rankings.	Implemented (vector channel not yet exercised end to end)
FR-9	Answer a natural-language question from stored memory, showing a source for each sentence.	Implemented
FR-10	Produce hour, day, week and month summaries and a morning brief, each citing the items it came from.	Implemented
FR-11	List open loops and propose completion only when evidence supports it, never closing anything automatically.	Implemented
FR-12	Draft a contextual reply and insert it into the field the user was typing in, after explicit confirmation.	Implemented
FR-13	Run model inference in a separate operating-system process that cannot take the	Implemented

ID	Requirement	Status in this work
	application down with it.	
FR-14	Select a GPU backend automatically at run time and fall back in a defined order, ending at the CPU.	Implemented
FR-15	Let the user pause all capture instantly and start the application in the paused state.	Implemented
FR-16	Download model files only after an explicit user approval, and only when the file matches a pinned digest.	Implemented
FR-17	Expose a diagnostics view containing counts and states but no captured content.	Implemented
FR-18	Capture meeting audio and produce live translation.	Not implemented (planned, Stage 8)

Requirements FR-1 to FR-17 are implemented and are the subject of Chapters 4 to 6. FR-18 is specified but not implemented, and is discussed in Section 7.8 as future work. Throughout this thesis, a requirement marked *implemented* means that the code exists and is exercised by tests; it does not by itself mean that the associated quality target in Table 3.2 is met.

3.4 Non-Functional Requirements

The non-functional requirements were written as numeric gates before implementation began, so that the evaluation could not be adjusted to fit the outcome. Table 3.2 lists each requirement together with its target and the outcome measured in Chapter 6; it therefore serves both as a specification and as a summary of results.

Table 3.2: Non-functional requirements and their measured outcome

ID	Requirement	Target	Measured outcome
NFR-1	No outbound network connection from the application except approved model downloads.	Zero connections over 24 h	Application process: zero in a 10-minute monitored run. The WebView2 runtime made one HTTPS connection (Section 6.4.3). Not met as written.
NFR-2	No captured text readable on disk without the user's Windows login.	Zero plaintext hits	Zero hits across 285 files / 32 MB after a 60-second run; positive control detected.
NFR-3	No screen image ever written to disk.	Zero image files	No image-writing code path exists; the file-

ID	Requirement	Target	Measured outcome
			system watcher test is still outstanding.
NFR-4	A text read must never block the capture loop.	≤ 300 ms wall clock	Maximum 294 ms over 16 windows and 9 applications.
NFR-5	Secrets removed before storage.	100 % of corpus; ≤ 2 % false positives	355 / 355 removed; 1 / 242 clean lines flagged (0.41 %).
NFR-6	Interface responsiveness.	60 fps; input-to-visual ≤ 100 ms	Frame p95 16.8 ms; input-to-visual p95 64 ms.
NFR-7	Interactive answer latency.	First token ≤ 2.5 s, full answer ≤ 10 s	25.2 s / 25.5 s on the available integrated GPU. Not met.
NFR-8	Draft latency.	First text ≤ 3 s, complete ≤ 8 s	5.9 s / 16.4 s on the available integrated GPU. Not met.
NFR-9	Extraction quality.	F1 ≥ 0.70; hallucination ≤ 3 %	F1 0.533; hallucination 0.0 %. Partially met.
NFR-10	The interface must not appear in screen recordings or shared screens.	Absent from captures	100 % absent from Windows Graphics Capture and GDI frames; named conferencing applications not yet exercised.
NFR-11	Background resource use.	CPU ≤ 3 %, RAM ≤ 250 MB over 8 h	Not measured; the 8-hour run has not been performed.
NFR-12	The application must build without any GPU vendor SDK installed.	Clean build	Verified; continuous integration builds the CPU-only path.

Six of the twelve targets were met, four were missed, and two were not measured. The reasons are analysed in Chapter 6 and discussed in Chapter 7. Two general observations are worth making here. First, every target that depends only on deterministic code was met. Second, every target that depends on the quality of a four-billion-parameter quantised model, or on the throughput of an integrated GPU, was missed. That separation is the most informative result in this thesis.

3.5 Use Cases

The use-case diagram appears as Figure 3.1 in Section 1.6. Three cases are described in detail here because they drive the architecture.

UC-1: Continuous capture. The user works normally. On each change of foreground window or title, and otherwise at a configurable cadence, the system checks a set of

gates, then attempts to read text from the active window, then redacts it, then merges it into an in-memory session. When attention moves elsewhere for thirty seconds, or after five minutes of continuous use, or when the accumulated text passes six thousand characters, or on session lock or shutdown, the session is written to the database as a single item. *Pre-condition:* capture is not paused — and capture begins paused on first run, so this is an action the user must take. *Alternative flow:* if any gate rejects the window, nothing is captured and a content-free audit event records the transition.

UC-4: Ask a question with sources. The user types a question. The system parses time, application and fact-kind filters from the question, runs the retrieval channels described in Section 4.9, selects up to eight snippets within a budget of roughly 1,500 tokens, and asks the model to answer using only those snippets, citing a snippet identifier for each sentence. As sentences stream back, each is checked against the snippet it cites. Verified sentences are shown with a citation chip that opens the underlying item; unverified sentences are hidden and counted. If nothing is verified, the system says it could not find the answer and offers the top three sources. *Post-condition:* the user is never shown an uncited claim.

UC-7: Contextual draft. With the cursor in a reply field, the user invokes the draft assistant. The system records the focused element's identity, checks that it is editable and is not a password field, reads the surrounding thread, retrieves relevant memory, and generates a draft at interactive priority. An independent checker then verifies every date, time, amount, number, email address, file name and personal name in the draft against the thread and the stored memory, flagging anything unsupported. The draft appears with its flags; nothing is inserted until the user presses Insert. On insertion the system re-verifies that the focus target is still the same element, saves every format of the current clipboard, writes the draft marked to be excluded from clipboard history and cloud synchronisation, sends the paste keystroke, and restores the clipboard. *The system never presses Enter.*

3.6 Constraints

Platform. Text acquisition, key protection, clipboard handling and window management depend on Win32, UI Automation and WinRT, so capture is Windows-only. The pipeline, storage, retrieval and guard libraries are deliberately platform-

neutral and are compiled and tested without Windows, which is also how they are exercised in continuous integration.

Hardware. The reference machine specified in the plan was not available. All model-dependent measurement was performed on an AMD Ryzen 5 7530U with Radeon integrated graphics, which shares system memory with the processor and has far lower bandwidth than a discrete card. This constrains the validity of every latency figure in this thesis.

No network. Because the system may not make outbound connections, several conveniences are unavailable: no model can be fetched automatically, no dependency advisory database can be refreshed during a build, and no error reporting exists. Each of these had to be replaced with something offline — pinned digests, a cached advisory database, and a content-free diagnostics bundle that the user inspects before saving.

Single developer, fixed schedule. The project was carried out by one student within a single semester, which is the principal reason that the vision rung, the embedding channel and several real-hardware verifications are implemented but not fully exercised.

3.7 Hardware and Software Requirements

Table 3.3: Hardware and software requirements

Category	Requirement
Operating system	Windows 11, 64-bit (the capture, clipboard and key-protection code uses Win32, UI Automation and WinRT interfaces).
Processor	x86-64 with AVX2; used for the embedding model and as the inference fallback.
Memory	8 GB minimum; 16 GB recommended when the vision-language model is resident.
Graphics (optional)	Any GPU with a CUDA, ROCm, Vulkan 1.2+, SYCL, Metal or OpenCL driver. Without one, inference runs on the CPU.
Disk	Approximately 4 GB for the quantised model and projection file, plus the growing encrypted database.
Rust toolchain	Stable MSVC toolchain, edition 2024 (rustc 1.99 or newer).
C/C++ toolchain	Visual Studio Build Tools with the C++ workload, required to compile the vendored OpenSSL used by SQLCipher.

Category	Requirement
Perl	Strawberry Perl on PATH, required by the OpenSSL build.
Node.js and pnpm	Node.js 24 or newer with pnpm 12.15.0, for the interface bundle.
Python	Python 3.11 or newer, for the evaluation harness and the dependency checker.
Inference runtime	A pinned llama.cpp build (tag b11900) fetched by the user for the chosen backend.
Models	Qwen3-VL-4B-Instruct Q4_K_M with its F16 projection file, and multilingual-e5-small in ONNX form, each fetched only after explicit approval.

3.8 Assumptions

16. The user's Windows account is not already compromised. The data-protection key is bound to that account, so an attacker who controls it can decrypt the database. This is stated as a limitation in Section 7.5 rather than defended against.
17. The machine is physically secure while unlocked. Capture stops on session lock, but an unlocked machine with the interface open exposes whatever is on screen.
18. Captured text is in a language the model handles. The embedding model is multilingual; the extraction prompts and the injection classifier are written for English and, partially, for transliterated Indian-language text. Section 6.5 shows that multilingual attacks are among the defence's weakest categories.
19. The user supplies their own model files after an explicit approval step. No model is bundled.

3.9 Scope Boundaries and Non-Goals

The following are explicitly out of scope: universal hardware compatibility, since a GPU without any of the supported drivers falls back to the processor; autonomous action of any kind; cloud synchronisation, accounts or telemetry; multi-user operation; capture on operating systems other than Windows; and defence against an attacker who already controls the user's Windows account.

3.10 Requirement Traceability

Each functional requirement is realised by an identifiable part of the system and verified by an identifiable measurement. FR-1 and FR-2 are realised by the capture ladder and gate logic of Section 4.6 and verified in Section 6.3; FR-3 by the redaction

engine of Section 5.5 and verified in Section 6.4.1; FR-4 and FR-6 by the storage schema of Section 4.8 and verified in Sections 6.4.2 and 6.6; FR-5 and FR-7 by the extraction pipeline and injection guard of Sections 5.9 and 5.10 and verified in Sections 6.5 and 6.6; FR-8 and FR-9 by the retrieval design of Section 4.9 and verified in Section 6.7; FR-10 and FR-11 by the rollup and open-loop logic of Section 5.13; FR-12 by the draft assistant of Section 5.12 and verified in Section 6.8; FR-13 and FR-14 by the process architecture and backend selection of Sections 4.5 and 4.7 and verified in Section 6.10; FR-15 by the gate logic; FR-16 by the network boundary of Section 5.15; and FR-17 by the diagnostics view.

CHAPTER 4 — SYSTEM ARCHITECTURE AND DESIGN

4.1 Overview

Tessera is a Rust workspace of ten libraries behind a desktop shell, running as two cooperating operating-system processes plus the interface renderer. This chapter describes how the parts fit together and why the boundaries fall where they do. Implementation detail — the algorithms, the data structures, the specific code — is deferred to Chapter 5.

4.2 Design Principles

Four principles determined most of the structure.

The model proposes, code disposes. No guarantee offered to the user may depend on the language model behaving correctly. Wherever the model produces something, deterministic code checks it before it has any effect. This principle is why the extraction pipeline has four validation stages after the model call, and why the answer pipeline verifies each sentence individually.

Fail closed. When a check cannot reach a conclusion, the system takes the restrictive branch. A browser window whose private-browsing state cannot be read is treated as private and skipped. A focused element whose password status cannot be determined is treated as a password field and refused. A model output that is partly invalid is rejected in full rather than repaired, because a repaired output is one nobody specified.

Make the dangerous thing structurally impossible. Where a rule can be replaced by an arrangement in which breaking it is not expressible, it is. All network code lives in one library, and a build-time check fails the compilation if any other library can reach a network-capable dependency. Sensitive strings are wrapped in a type whose display implementation prints `<redacted>`, so a careless log statement cannot leak content. A fact cannot exist in the database without an evidence row, because a trigger deletes it when the last one is removed.

Measure against criteria written beforehand. Every numeric target in Table 3.2 was recorded before the corresponding code was written, and the evaluation harness is part of the repository.

4.3 High-Level Architecture

Figure 4.1 shows the system as a whole.

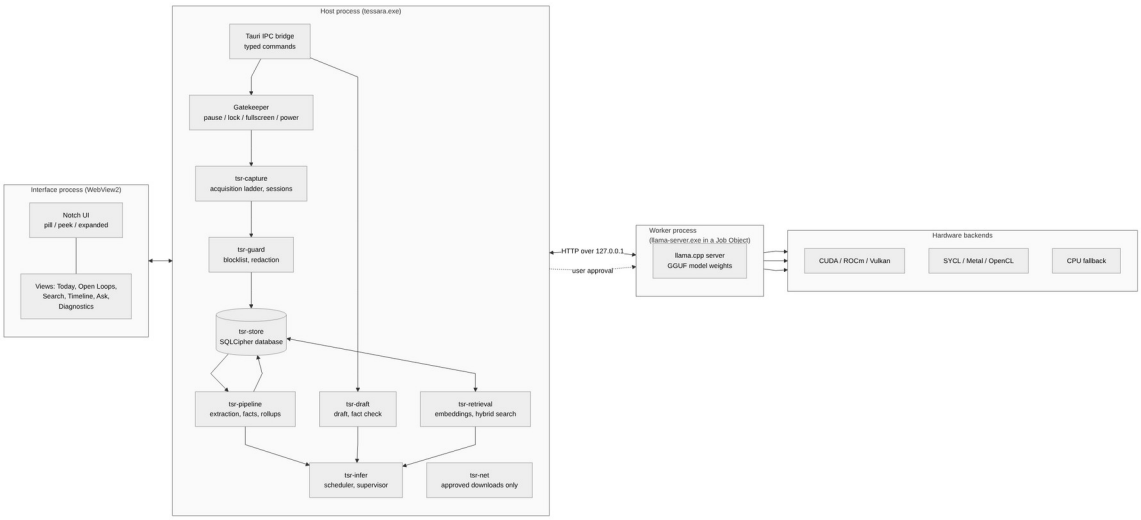


Figure 4.1: High-level architecture. Three process boundaries separate the interface, the orchestration host and the inference worker; the hardware backend is chosen at run time.

Three process boundaries divide the system. The **interface process** is the WebView2 runtime rendering a React application; it holds no data of its own and communicates only through a fixed set of typed commands. The **host process** contains all of the application's own logic: gating, capture, redaction, storage, retrieval, the pipeline, scheduling and supervision. The **inference worker** is an external `llama-server` executable confined to a Windows Job Object, reachable only over a loopback socket.

The arrangement is deliberately asymmetric in trust. The host process trusts neither of the other two. It validates everything arriving from the interface, and it treats everything arriving from the worker as an unverified proposal.

4.4 Module Decomposition

The host process is divided into ten libraries. The division follows the trust and platform boundaries rather than convenience: the libraries that implement security decisions are platform-neutral, so they can be tested exhaustively without a Windows desktop, and the one library that can open a network connection is isolated so that a tool can prove nothing else does.

Table 4.1 lists each library, what it is responsible for and its size in Rust source lines. The whole system is written in Rust, whose compile-time memory and thread-safety

guarantees [26] matter here because the capture path handles untrusted text on background threads.

Table 4.1: Workspace crates and their responsibilities

Crate	Responsibility	Rust source (lines)
tsr-core	Domain primitives, the Sensitive<T> wrapper that prints as <redacted>, and the allowlist-based content-free logging layer.	475
tsr-win	Safe wrappers over Win32 and WinRT: DPAPI, Job Objects, display affinity, window event hooks, clipboard, TCP table queries.	1,771
tsr-capture	The three-rung text acquisition ladder, the deadline worker, change detection and session aggregation.	5,101
tsr-guard	Blocklist evaluation, the redaction engine and the claim checker. Platform-neutral, so it compiles and tests without Windows.	1,651
tsr-store	SQLCipher database, forward-only migrations, full-text indexes and the repository layer.	3,081
tsr-retrieval	Pinned ONNX Runtime loading, embedding, chunking, the hybrid search engine and the Ask pipeline.	1,621
tsr-infer	Adapter discovery, GGUF header parsing, backend selection, the sidecar supervisor, the loopback HTTP client and the priority scheduler.	8,664
tsr-net	The only network-capable crate: approval-gated, resumable, digest-verified model downloads.	1,115
tsr-pipeline	Extraction, schema parsing, evidence verification, the injection guard, date and people resolvers, rollups, briefs and open loops.	7,254
tsr-draft	Draft generation, the independent fact checker and the draft state machine.	587
src-tauri	The desktop shell: window placement, tray, hotkeys, typed IPC commands and the view	3,138

Crate	Responsibility	Rust source (lines)
	layer.	
evals/runner	The tsr-eval harness that runs the extraction, Ask, draft, injection and backlog evaluations.	2,331
bench/uia-probe	The standalone probe used to measure text acquisition against real applications.	407

Three aspects of this decomposition are worth remarking on. First, `tsr-guard` contains the blocklist and redaction engines and has no Windows dependency at all, which is what allows its 355-case secrets corpus to run in ordinary continuous integration. Second, `tsr-infer` is the largest library because it carries four separable concerns — hardware discovery, process supervision, transport and scheduling — that nevertheless share a lifecycle. Third, `tsr-net` is small by design: it is the whole attack surface for outbound traffic, so it is kept small enough to read.

4.5 Process Architecture and Isolation

Figure 4.2 traces an interactive request from the interface to the hardware and back.

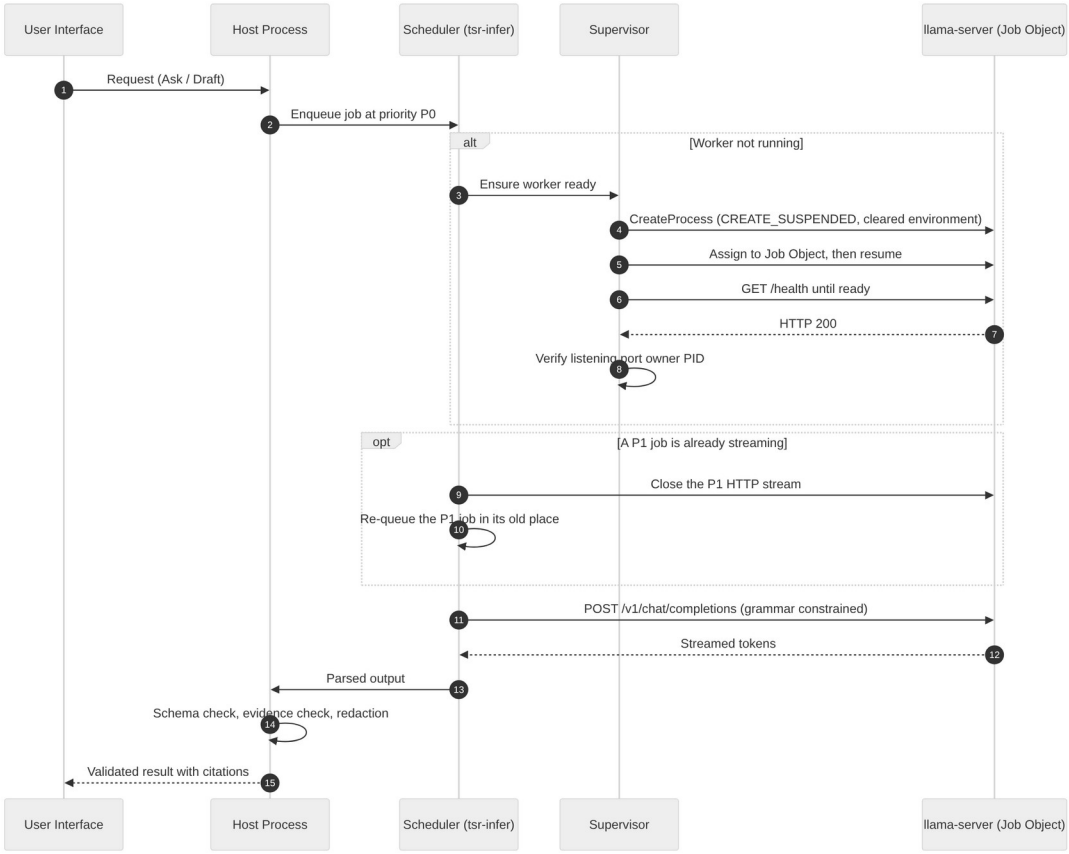


Figure 4.2: Sequence of an interactive inference request, including worker start-up and the preemption of a background job.

The inference worker is started through a specific sequence, each step of which closes a particular failure mode.

20. The child is created **suspended**, so that it cannot execute a single instruction before the remaining controls are in place.
21. Its **environment is cleared** and repopulated from a short allowlist (`SystemRoot`, `windir`, `TEMP`, `TMP`, `NUMBER_OF_PROCESSORS`). The inference server reads configuration from environment variables with an `LLAMA_ARG_` prefix, so an inherited variable could otherwise change its behaviour — for instance by enabling a feature or supplying a credential. This is not hypothetical: it was tested by setting such a variable in the parent and confirming that the child ignored it.
22. The child is **assigned to a Job Object** configured to kill its members when the last handle closes. Because that handle closes when the host process exits for any reason, an orphaned worker holding gigabytes of GPU memory cannot survive a crash.

23. The child is then **resumed**, binds a random high-numbered port on the loopback interface, and the host polls its health endpoint.
24. Before trusting that endpoint, the host **verifies the port's owner**: it queries the operating system's TCP table and confirms that the process identifier owning the listening socket is the child it created. Without this check, any other local process could race to bind the port and impersonate the worker.

A supervisor state machine then manages the worker's lifetime: health polling, hang detection, exponential backoff with jitter on restart, a restart budget so that a permanently broken configuration is reported rather than retried forever, and unloading the model after an idle period so that it does not occupy memory while the user is doing something else.

4.6 Data Acquisition Design

Figure 4.3 shows the acquisition path end to end.

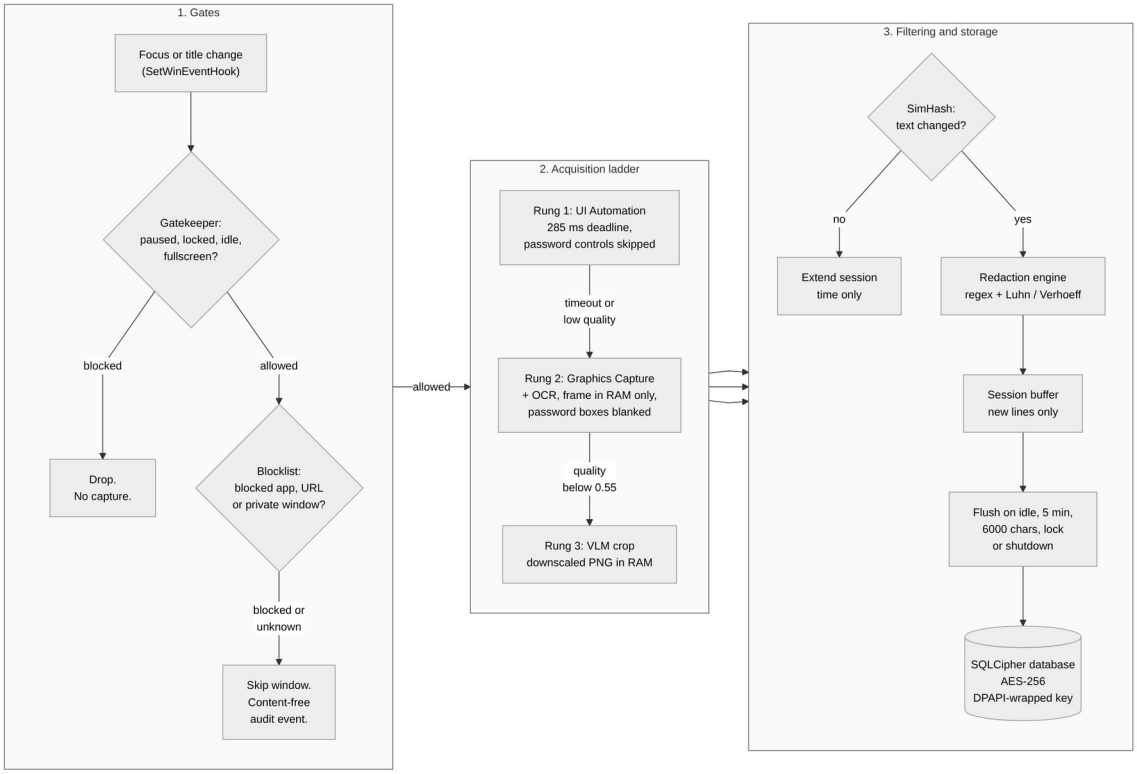


Figure 4.3: Data acquisition and privacy pipeline. Each gate can only reduce what is captured; nothing bypasses redaction on the way to storage.

Acquisition is organised as a sequence of gates followed by a three-rung ladder. The ordering matters: every gate that can reject a window runs before any text is read, and redaction runs before anything reaches a buffer that might later be written.

Gates. Capture begins paused on first run. Beyond that, the gatekeeper stops capture when the session is locked, when the machine is suspending, when the user has been idle, and when a full-screen Direct3D application or a presentation is in the foreground — the last of these so that capture cannot drop frames in a game or a presentation. On battery power the sampling interval is multiplied and the vision rung is disabled entirely.

Blocklist. Surviving windows are evaluated against process names, window titles and browser private-state indicators. Password managers and banking sites are blocked by default. A browser window whose private state cannot be determined is blocked. A deliberate refinement here is that generic words do not over-block: a non-browser window merely containing the word "password" in its title is still captured, because otherwise a developer's text editor would be invisible to the system; this behaviour is pinned by a dedicated test.

Rung 1 — UI Automation. The accessibility tree of the focused application is read on a dedicated thread with a 285-millisecond wall-clock deadline. Controls marked as password fields are excluded structurally rather than filtered afterwards. The deadline is the central design element here: accessibility calls are synchronous requests into another process and can block for seconds, so a worker thread performs the read and the caller abandons the result when the deadline expires. The abandoned read continues in the background and its result is discarded. In measurement across sixteen windows and nine applications, no read ever blocked the caller for more than 294 milliseconds, although the abandoned reads themselves ran for as long as 732 milliseconds.

Rung 2 — Graphics capture with optical character recognition. When rung 1 times out or returns too little, a single frame is taken into an in-memory bitmap and passed to the WinRT recognition engine. Because a bitmap has no structure, the bounding boxes of known password controls are blanked before recognition.

Rung 3 — Vision model. When recognition quality falls below a configurable threshold, the window is downscaled to at most 1280 pixels, encoded to a PNG in memory and sent to the vision-language model. At no point is an image written to disk.

Change detection and sessions. The resulting text is fingerprinted with a 64-bit SimHash. If the fingerprint is within a Hamming distance of three of the session's current fingerprint, only the session's end time is extended. Otherwise, only the lines the session has not already seen are appended, so that a chat window scrolled through repeatedly contributes each message once.

Redaction, then storage. Redaction runs on the text, the window title and the URL before any of them enters the session buffer. Sessions are flushed to the database as single items on the conditions described in Section 3.5. Pausing discards open sessions without storing them.

4.7 Hardware Backend Selection

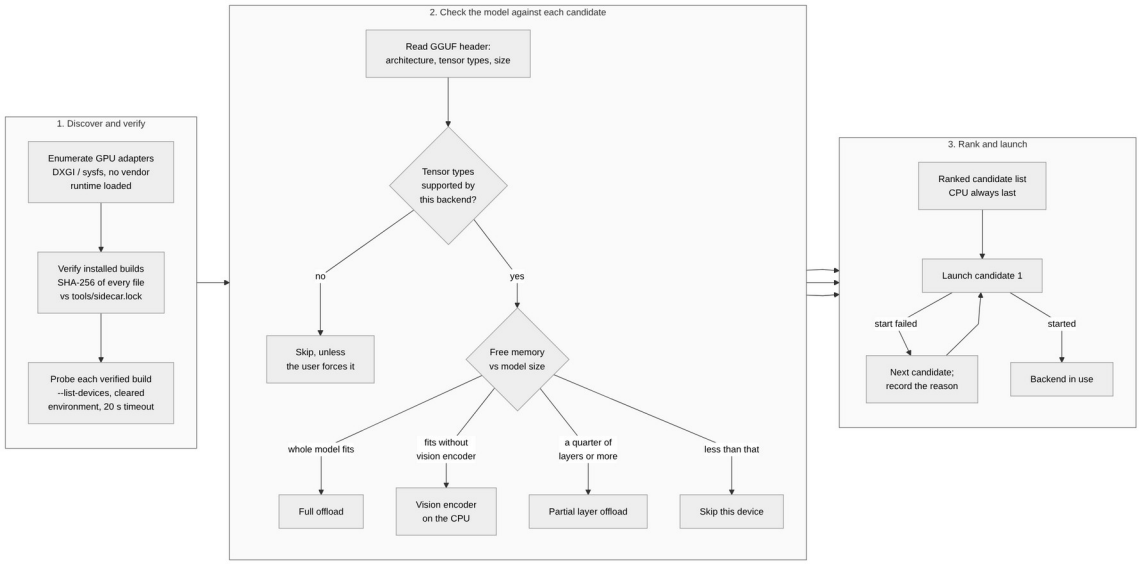


Figure 4.4: Backend selection and fallback. Detection, verification, probing and model inspection all precede any attempt to launch a worker.

The design goal is that the application binary contains no vendor code, and that the choice of backend is a run-time discovery rather than a build-time assumption. The six-step arbitration is shown in Figure 4.4.

Three aspects deserve emphasis. **Adapters are enumerated without loading a vendor runtime** — through DXGI on Windows and the kernel's device tree on Linux — because loading a vendor runtime on a machine with a mismatched driver is itself a common cause of failure. **Each candidate build is verified byte-for-byte** against a manifest compiled into the application, covering every file and refusing any extra file, because the acceleration code lives in a separate library beside the server executable

and pinning only the executable would leave the interesting part unpinned. And **the probe is real**: each verified build is executed with a device-listing flag in a cleared environment, which initialises the backend for genuine and reports usable devices and free memory without loading any model.

The model is never altered to fit the hardware. If a tensor type is not known to be supported by a backend, that backend is skipped rather than attempted; if the model does not fit, the system offloads fewer layers or moves the vision encoder to the processor, but it does not re-quantise or substitute a different model. The supported backends and their verification status are given in Table 4.2.

Table 4.2: Inference backends, the hardware they serve, and their verification status in this work

Backend	Hardware served	Automatic order for that vendor	Verification status
CPU	Any x86-64 with AVX2, or ARM64	Always the final fallback	Verified on real hardware
Vulkan	AMD, Intel, NVIDIA, Qualcomm	Second choice for every GPU vendor	Verified on real hardware (AMD Radeon integrated GPU)
CUDA	NVIDIA	First choice on NVIDIA	Selection logic verified against simulated adapters only
ROCm (HIP)	AMD discrete, RDNA2 and newer	First choice on AMD	Selection logic verified against simulated adapters only
SYCL	Intel Arc and Xe	First choice on Intel	Selection logic verified against simulated adapters only
Metal	Apple Silicon	Only choice on Apple	Selection logic verified against simulated adapters only
OpenCL	Qualcomm Adreno	First choice on Qualcomm	Selection logic verified against simulated adapters only
OpenVINO	Intel GPU and NPU	Never chosen automatically; used only when forced	Selection logic verified against simulated adapters only

4.8 Storage Design

Figure 4.5 shows the entities and their relationships, and Table 4.3 lists the tables each migration creates.

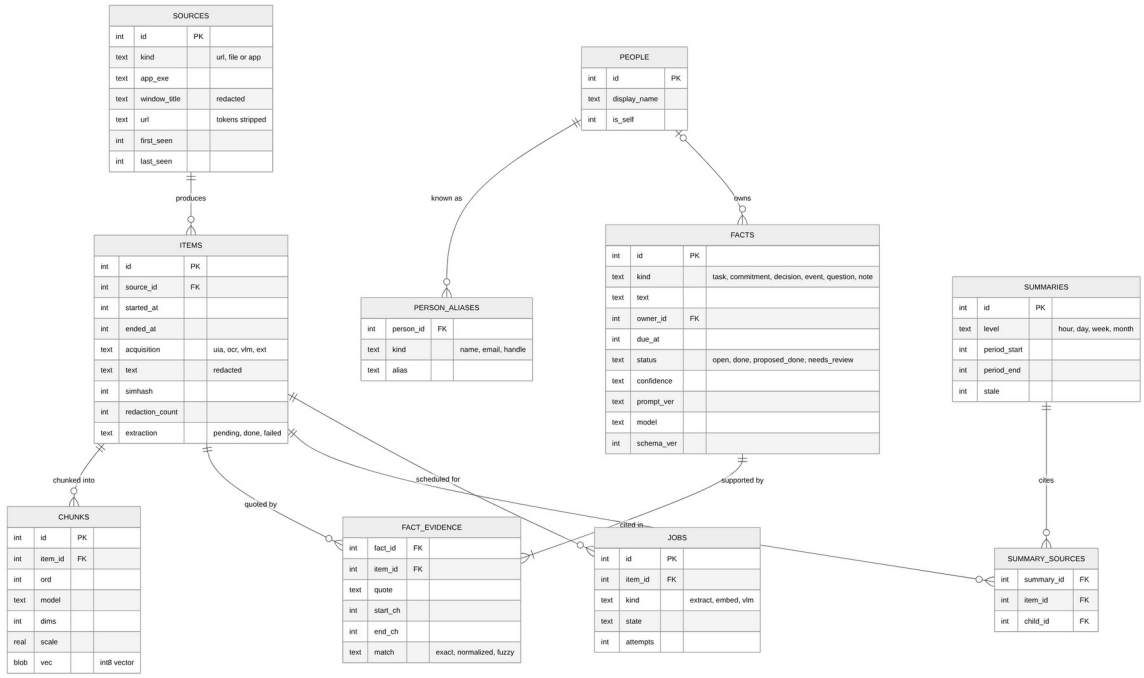


Figure 4.5: Entity-relationship diagram of the encrypted store, as created by the four forward-only migrations.

The schema is created by four forward-only migrations tracked in the database's own version counter, each applied inside a transaction together with its version bump so that a failure leaves the database at the previous version. A database written by a newer build is refused rather than guessed at.

Table 4.3: Database tables created by the four forward-only migrations

Migration	Tables and indexes created	Purpose
0001	blocklist, settings, audit_log	Capture policy, application settings and the content-free audit trail.
0002	sources, items, items_fts (+ three synchronisation triggers)	What was read, from where and when, with the full-text index kept in step by triggers.
0003	people, person_aliases, person_merge_suggestions, suppressed_aliases, facts, fact_evidence, facts_fts, jobs, chunks	Identity resolution, structured facts with their evidence spans, durable job state and embedding vectors.
0004	summaries, summary_sources, briefs	Hour, day, week and month rollups with provenance, and the assembled morning brief.

Several decisions in the schema exist to enforce guarantees that would otherwise be left to application code.

Provenance is mandatory. A fact is meaningless without evidence, so `fact_evidence` carries the quotation, its character span in the item, and how it was matched — exactly, after normalisation, or fuzzily. A trigger deletes the fact when its last evidence row is removed, which means that deleting a captured item automatically removes any fact that depended solely on it.

Enumerations and lengths are enforced by the database. Fact kinds, statuses, confidences, acquisition rungs and job states are all constrained by `CHECK` clauses, as are text lengths. A bug in the pipeline therefore cannot store a fact in a state no one designed.

Full-text indexes are kept in step by triggers, and deletions are honoured immediately. Both full-text indexes are configured so that deleted terms are removed from the index at once rather than at the next merge, so that a deleted memory is not still discoverable through index pages.

Forgetting is one-way. When a user asks the system to forget a person, the suppression list stores only a keyed digest of the normalised identifier, salted per database. The list therefore cannot be read back into a set of names.

Jobs are durable. The scheduler itself is in memory; the `jobs` table is what survives a restart, so queued and running work is re-queued on the next start rather than lost.

4.9 Retrieval Design

Retrieval runs three channels and fuses them. The lexical channel runs BM25 over captured items; a second lexical channel runs over the facts table and maps each hit back to the items its evidence points into, so that a well-phrased fact can surface an item whose raw text matches poorly; the vector channel embeds the query and performs a brute-force cosine comparison against the stored int8 chunk vectors.

Two design choices are worth justifying. First, the vector search is **brute force** rather than an approximate index. At the scale of a single user's memory this is fast enough, and it removes an entire category of correctness problem — an approximate index that silently degrades as it grows — in exchange for a cost that is linear and predictable. Second, **filters are applied before fusion, identically on every channel**: a source must pass the *current* blocklist, not the blocklist that was in force when it was captured, and an item mentioning a suppressed identity is excluded everywhere.

Applying the filter after fusion would allow a newly blocked source to influence ranking even if it were not shown.

Fusion uses Reciprocal Rank Fusion with the conventional constant of 60, after which a recency multiplier of 1 + 0.15 × 0.5^(age in days / 14) is applied. The multiplier is deliberately weak: it can reorder near-ties but cannot outweigh a channel's judgement. Ties break by newer item and then by lower identifier, so that the ordering is fully deterministic and therefore testable. If the vector channel is unavailable, the keyword results are returned alone and the diagnostics record that this happened, rather than silently returning a worse result.

4.10 Scheduling Design

Figure 4.6 illustrates what happens when the two collide.

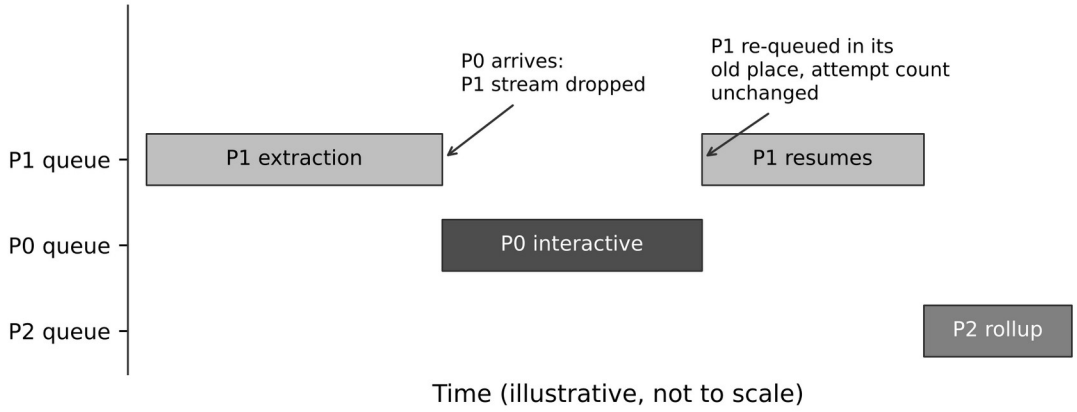


Figure 4.6: Scheduler behaviour when an interactive request arrives while a background extraction job is streaming.

One model instance serves interactive requests, background extraction and periodic summarisation, which are activities with very different latency requirements. The scheduler assigns three priorities: P0 for interactive work such as answering a question or writing a draft, P1 for near-real-time work such as extraction, and P2 for background work such as rollups.

Because the inference server processes one request at a time, a background job already streaming would otherwise delay an interactive one by many seconds. The scheduler therefore **preempts by dropping the connection**: closing the HTTP stream causes the server to abort generation immediately. The preempted job is re-queued *in its original position* with its attempt count unchanged, so that preemption is not

treated as a failure, and partial output is discarded because nothing is committed until a job completes.

Three further rules prevent pathologies. A job may be preempted only a bounded number of times, after which it runs to completion, which prevents livelock. A background job that has waited beyond a threshold is promoted to compete at the next priority up, which prevents starvation. And a job key may be queued or running only once and is refused after completion, which makes duplicate submissions and retries idempotent — a job can never complete twice.

The scheduler is written as a pure state machine that owns no threads, never sleeps, and takes the current time as an argument. The same code therefore runs both in the threaded runtime and inside a discrete-event simulation used for evaluation, which is what makes its behaviour reproducible.

4.11 Privacy and Security Boundaries

Table 4.4 lists the trust boundaries, what crosses each one and the control applied there.

Table 4.4: Trust boundaries in the system

Boundary	What crosses it	Control applied
Interface process to host process	Typed command names and identifiers	Fixed command set, validated lengths, enumerations and identifiers; errors carry a code, never content.
Host process to inference worker	Prompt text and generated tokens	Loopback address only; the listening socket's owning process identifier must equal the spawned child's.
Application to database	Captured text, facts, summaries	Whole-database encryption; the key is unwrapped from the Windows login and wiped from memory after use.
Application to the internet	Model file bytes	One crate holds all network code; it requires a typed approval value and verifies a pinned digest.
Captured screen text to the model prompt	Untrusted text	Escaped and wrapped in delimiters; the model has no tools; its output is constrained by a grammar and re-validated in code.
Model output to storage	Proposed facts	Schema check, evidence must exist in the source, injection classification, second redaction

Boundary	What crosses it	Control applied
		pass.

The boundary between captured text and the model prompt deserves particular attention, because it is where the injection problem lives. Captured text is escaped and placed inside explicit delimiters, and the prompt states that the delimited region is data. That alone would be a weak defence, since a sufficiently persuasive injection can talk its way past an instruction. The defence that actually carries the weight is the absence of capability: the model has no tools, cannot make requests, and its output is constrained by a grammar to a JSON object with a fixed set of fields. A successful injection can therefore influence what text is proposed, and nothing else. Sections 5.10 and 6.5 describe the additional classifier that attempts to stop even that, and measure how often it fails.

Logging is subject to a complementary discipline. Rather than filtering content out of log messages, the logging layer admits only messages and fields on an allowlist, and discards any field value that is not numeric, boolean or a short allowlisted identifier. A developer adding a log line with a user string therefore produces no output rather than a leak.

4.12 Error Handling and Recovery

Failures were categorised during design and each given a defined behaviour rather than being handled where they happened to arise.

A **worker crash or hang** is detected by the supervisor, which restarts with exponential backoff and a restart budget; in-flight jobs are re-queued and the interface reports a degraded state. A **backend that fails to start** causes the next candidate to be tried, with the reason recorded; the processor is always the last candidate. A **text read that exceeds its deadline** falls through to the next rung. A **model output that fails schema validation** is retried once and then the item is marked as failed, rather than being partially parsed. A **fact whose evidence cannot be located** is dropped with a counted reason. A **database that is newer than the build** causes a refusal to open. A **migration failure** leaves the database at its previous version because each migration shares a transaction with its version bump. Every one of these produces a content-free

audit event, so that the diagnostics view can show what went wrong without showing what the user was doing.

4.13 User Interface Design

The interface is a single fixed-size transparent window pinned to the top of the monitor under the cursor, with three states: a compact pill, a peek showing a short summary, and an expanded panel containing the Today, Open Loops, Search, Timeline, Ask and Diagnostics views. Switching panels changes the window *region* rather than resizing the window, which avoids the visual artefacts that resizing a transparent layered window produces.

Two behaviours are design requirements rather than refinements. The window sets display affinity [23] to exclude itself from capture, on every visible window of the process, and reads the value back to confirm it took effect; if it did not, the interface says so rather than silently appearing in a screen share. And the interface does not steal focus: a peek appears without taking the foreground, and when a user-opened panel closes, focus returns to the application that had it, so that keystrokes never vanish into a hidden window.

CHAPTER 5 — IMPLEMENTATION AND METHODOLOGY

5.1 Development Methodology

The project was developed in numbered stages, each with written exit criteria fixed before the work began. Stage 0 established feasibility and tooling; Stage 1 built the foundation, capture and baseline guardrails; Stage 2 the understanding pipeline; Stage 3 the interface; Stages 4 to 6 the rollups, draft assistant and question answering; Stage 7 a security audit. Stages 8 to 10 — meeting capture, closed beta and release engineering — were gated behind the release criteria and are not implemented.

Two practices from this method are worth describing because they shaped the result.

Decisions were recorded, including the uncomfortable ones. Ten architecture decision records document the significant choices, each stating the context, the options considered and the consequences. Two of them record unresolved problems rather than solutions: one records an outbound network connection made by the WebView2 runtime that the project could not switch off, and one records that the automatic updater cannot ship because of unpatched advisories in a transitive dependency. Writing these down as open items, rather than quietly narrowing the criterion they violate, is what allows Chapter 6 to report a failure instead of a redefinition.

Table 5.1 lists the ten records and what each settles.

Table 5.1: Architecture decision records and the questions they settle

Record	Question settled	Outcome
ADR-0001	Which dependency versions and feature flags carry privacy guarantees?	Exact version pins; default features of the embedding crates disabled so that no HTTP client or build-time download enters the graph; the ONNX Runtime API level raised so that the system copy cannot be used.
ADR-0002	What to do about an outbound connection made by the WebView2 runtime?	Open. Recorded as a failure of the zero-network criterion rather than explained away.
ADR-0003	Should the automatic updater ship?	No. It is compiled out because a transitive compression library carries unpatched advisories.
ADR-0004	How is GPU-vendor independence achieved?	Backends live outside the binary as pinned, hash-verified

Record	Question settled	Outcome
		builds; the application detects, probes, ranks and falls back; the model is never silently changed to fit.
ADR-0005	What redaction thresholds are acceptable?	100 % recall on the seeded corpus with a false-positive ceiling of 2 % on clean text.
ADR-0006	How is the understanding pipeline staged and validated?	Durable, preemptible jobs; strict schema; evidence as source spans; a second redaction pass; a single atomic write.
ADR-0007	How is the floating interface placed and hidden?	One fixed-size transparent window with per-panel regions; display affinity set on every window and read back.
ADR-0008	How are rollups, drafts and Ask guarded against a compromised model?	Independent code-side checks: provenance validation, an independent claim checker, per-sentence citation verification.
ADR-0009	What did the security audit change?	Draft flagging for injected thread lines, a fallible date resolver, redaction run to a fixed point, model digest checking at start-up.
ADR-0010	Why were answers empty?	JSON schema keys were being serialised in alphabetical order, so the grammar forced the model to commit to an "insufficient" field before writing any answer. Declared key order was restored.

Verification was separated from implementation. Each stage produced a verification report labelling every measurement as REAL, SIMULATED or UNVERIFIED. A claim measured with a stand-in model is never presented as if it had been measured with a real one. This distinction is preserved throughout Chapter 6.

5.2 Repository Structure and Build

The workspace is a Cargo monorepo. Dependencies whose feature flags carry privacy consequences are pinned to exact versions rather than compatible ranges, and two of these pins are themselves security controls.

The embedding library's default features would pull in a client for a model hub — an HTTP stack — and a build-time download of the inference runtime. Both are disabled, and the runtime is loaded dynamically from a local file instead. The ONNX Runtime binding is configured to demand API level 31, which causes it to refuse any runtime library older than version 1.31. This matters because Windows ships its own copy of that library in the system directory, at version 1.17 on the development machine; without the API-level requirement the system copy would be silently accepted, and the "pinned runtime" guarantee would be false. A test confirms that the system copy is never mapped into the process.

```
cargo test --workspace -- --include-ignored     # 375 test functions
python tools/check_network_deps.py --locked     # dependency boundary
cargo deny check                                # licences, bans,
advisories
python evals/validate.py                        # evaluation set
integrity
pnpm build:ui && cargo build -p tessara --features custom-protocol
```

5.3 Core Primitives and Content-Free Logging

`tsr-core` defines `Sensitive<T>`, a wrapper for any string that may contain user content. Its `Debug` and `Display` implementations print `<redacted>`. The practical effect is that the ordinary ways content escapes — a debug print during development, a value interpolated into an error message, a panic message containing a captured string — all produce the placeholder instead.

The logging layer takes the complementary approach of an allowlist rather than a filter. A `tracing` subscriber emits one JSON line per event and keeps the message **only if it is one of a fixed, reviewed set of strings**; otherwise the line records a null message and names the call site so that the offending log statement can be found and fixed. Field values survive only if the field name is on a second list and the value is a number, a boolean, or a string of at most 160 printable ASCII characters. Everything else is dropped and counted.

This design means that adding a new log message is a reviewed change to a single file. A workspace-wide test parses every `tracing::` call in the repository and fails if any of them uses a message or field name that is not on the lists, so the allowlist cannot drift out of date.

5.4 Capture Implementation

The capture core is written as a platform-neutral state machine. The operating system is behind a `Platform` trait, storage is behind a `Sink` trait, and the current time is passed in as an argument. The consequence is that the whole of the gate logic, the ladder ordering, the blocklist interaction, session merging and flushing can be tested with simulated windows and a real encrypted database, with no desktop session and no clock, which is how the behaviour described in Section 6.3 is verified.

The deadline worker. Accessibility reads are the one part of capture that cannot be made safe by structure alone, because the call blocks inside another process. The implementation runs reads on a dedicated thread initialised for the multi-threaded apartment and has the caller wait with a 285-millisecond deadline. When the deadline expires the caller proceeds without a result and the worker's eventual answer is discarded. The measured consequence is that the capture loop never stalled beyond 294 milliseconds even against applications whose underlying reads took 732 milliseconds.

Change detection. SimHash is computed over overlapping groups of three consecutive words. An important implementation detail is that the hash function is a hand-written FNV-1a rather than the standard library's hasher, because the standard hasher is randomly seeded per process and its algorithm is not guaranteed stable across versions. The fingerprint is stored in the database and compared across runs, so it must be reproducible.

Sessions. A session is identified by the process, the top-level window and the document — the URL for a browser, and otherwise the redacted title with volatile counters such as an unread count removed, so that a changing message count does not look like a new document. When a snapshot's fingerprint matches, only the end time advances. When it does not, only the lines the session has not already recorded are appended.

5.5 Redaction Implementation

Each detector returns byte spans; overlapping spans are merged; each surviving span is replaced by a marker naming the kind of secret. Only per-kind counts leave the module — never the matched values, which is why a redaction count can appear in the database and in diagnostics without leaking anything.

Fifteen kinds are detected: private keys, JSON web tokens, API keys, payment cards, Aadhaar numbers, PAN, passport and voter identity numbers, bank account numbers, IFSC codes, UPI identifiers, one-time codes, passwords, generic secrets, and optionally phone numbers and email addresses. The last two are retained by default because identity resolution needs them, and can be redacted by configuration.

The design decision that determines the false-positive rate is that **numbers which could be anything are redacted only when a checksum agrees**. A sixteen-digit sequence is redacted as a payment card only if it satisfies the Luhn algorithm *and* carries a recognised network prefix. A twelve-digit sequence is redacted as an Aadhaar number only if it satisfies the Verhoeff check. Keys and tokens are matched by their published prefixes; an unprefixed high-entropy string is redacted only when it appears next to a word such as "key", "token" or "secret".

One defect found during the security audit is worth recording because it illustrates why a single pass is insufficient: a second one-time code falling inside the keyword window of the first survived redaction, because replacing the first changed the offsets. Redaction now runs repeatedly until the output stops changing.

5.6 Storage Implementation

The data directory holds two files: the encrypted database and a key file consisting of a four-byte magic value, a version byte and a data-protection blob. A key is created only when **neither** file exists. If the database is present but the key file is missing or unreadable, opening fails rather than generating a new key — because generating one would make the existing database permanently unreadable while appearing to succeed.

The database is opened with security-relevant pragmas: memory security on, secure deletion on, foreign keys enforced, temporary tables in memory, and write-ahead logging. SQLCipher's own logging is disabled, which is verified by a test that opens a database with the wrong key in a child process and asserts that nothing is written to standard error — a plausible leak path, since a cipher error message can contain fragments of the file.

5.7 Inference Sidecar Implementation

The loopback client is a minimal HTTP/1.1 implementation written directly on the standard library's networking, with no HTTP crate. This is not an exercise in reinvention: including a general HTTP client would put a network-capable dependency into the graph and break the compile-time boundary described in Section 5.15. The client is restricted to the loopback address, speaks only the few verbs the inference server needs, and supports the streaming responses the scheduler must be able to abort.

The GGUF parser reads only the header — the format version, architecture, tensor types and sizes — without touching tensor data, so inspecting a 2.4-gigabyte model file costs a few kilobytes of reads.

5.8 Scheduler Implementation

The scheduler is a pure state machine: it owns no threads, never sleeps, and receives the current time as a parameter. Two consumers drive it. The threaded runtime calls it from the application; a discrete-event simulation calls it from the evaluation harness to replay a synthetic backlog deterministically. Because both use the same code, a scheduling behaviour verified in simulation is the behaviour the application exhibits.

Each job carries a *key*, such as `extract:item:42`. A key may be queued or running only once, and a key that has completed is refused until it is explicitly forgotten. Retries and duplicate submissions therefore cannot cause a job to complete twice — which matters because completing an extraction twice would write a duplicate set of facts.

Preemption is implemented by cancelling the HTTP stream, which the inference server observes as a client disconnection and responds to by aborting generation. The preempted job returns to its original queue position with its attempt count unchanged, and any partial output is discarded.

5.9 Extraction and Deterministic Validation

Figure 5.1 shows the complete path from flushed text to a stored fact.

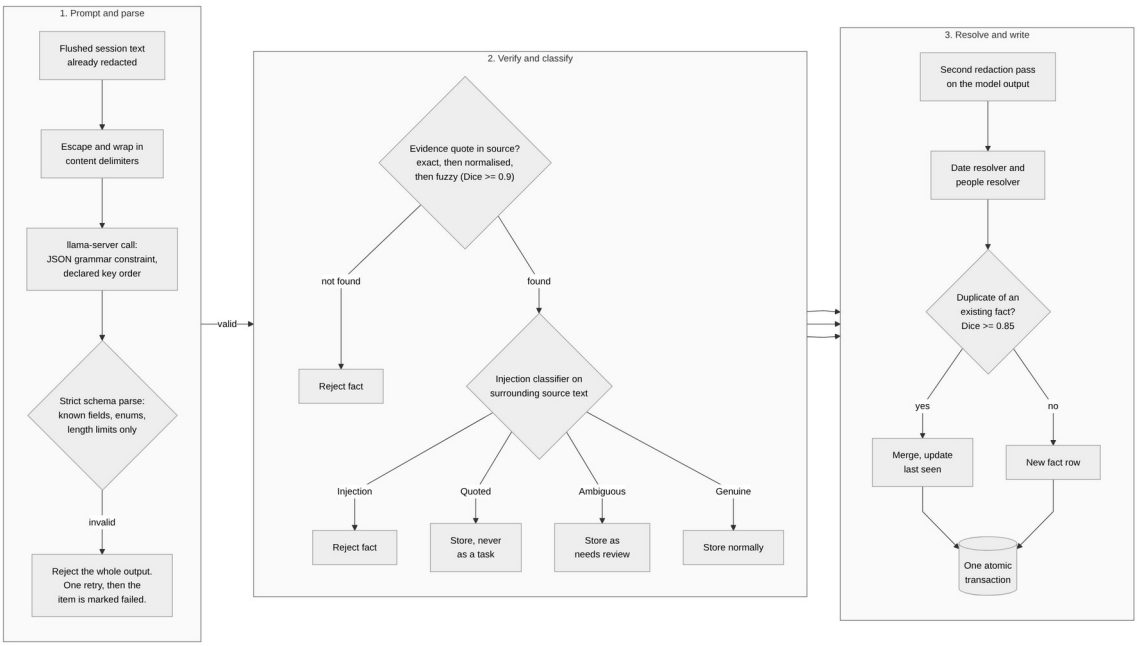


Figure 5.1: Extraction and deterministic validation. The model proposes; code decides what is stored, and in what state.

Extraction is the clearest expression of the principle in Section 4.2, so the stages are described in order.

Prompting. The flushed session text — already redacted — is escaped and wrapped in explicit delimiters. The model is given a JSON schema and constrained by a grammar so that the output cannot be prose, cannot be fenced in a code block and cannot contain unknown fields.

Schema parsing. The parser accepts exactly the schema: one JSON object, no unknown fields, enumerated values from fixed lists, length limits counted in characters, no control characters, no empty strings. Any violation rejects the whole output. It is deliberately **never repaired into something valid-looking**, because a repaired output is one that no one specified and whose relationship to the source is unknown.

Evidence verification. Each fact must carry a quotation. The verifier tries an exact substring match, then a match after normalising whitespace and punctuation, then a fuzzy match over word sets in a sliding window using the Dice coefficient with a threshold of 0.9 on quotations of at least four words. Quotations shorter than two words or eight characters are refused as too weak to support anything. Model text that is an explanation, a paraphrase or an invention therefore never becomes evidence. The

character span of the successful match is stored, which is what makes a citation chip able to open the item and highlight the right text.

Injection classification. Described separately in Section 5.10.

Second redaction. Redaction runs again on the model's output, because the model may reproduce a secret in a different form from the one the first pass matched.

Resolution. The date resolver converts the phrase the model copied into a timestamp *against the item's capture time, never against the current time* — so reprocessing an old item cannot silently change what "tomorrow" meant. Inputs are explicit: the reference time with its zone, the user's numeric date order taken from the Windows locale, and the end-of-day hour. Results are one of resolved, ambiguous, unresolved or invalid; an ambiguous result — a day-month ordering that could go either way, "next Friday", a weekday that does not match the stated date, or a time that falls in a daylight-saving gap — is stored with its preferred reading flagged as ambiguous and is never presented as certain.

The people resolver is deliberately conservative. An email address or handle identifies a person exactly. A bare name links to an existing person only if exactly one person has that name and no identifier contradicts it; the same name with a different email address is a different person. If two people share the name, the fact simply keeps the name as text. Near matches are *suggested* for the user to confirm and never merged automatically.

Deduplication and the atomic write. A fact sufficiently similar to an existing one is merged and its last-seen time updated. Everything — the facts, their evidence rows and the item's summary — is written in a single transaction.

5.10 The Injection Guard

The guard classifies each proposed fact from **the source text surrounding its evidence**, not from the model's wording. This is the key design decision: a model that has been successfully injected will word its output innocuously, so judging the output alone would be judging the attacker's choice of words.

Four classes are produced. *Injection* — text aimed at an assistant or model, or asking to exfiltrate, delete or reveal data, or to edit the system's own memory — rejects the

fact outright. *Quoted* — an instruction inside a quotation, a forwarded block or a code block — permits storage but never as a task. *Ambiguous* — an imperative addressed to "me" with no sign that it is addressed to the user — is stored with a status that keeps it out of task lists until the user confirms it. *Genuine* covers everything else.

The classifier is conservative by design, to the point that a legitimate note which merely quotes an injection phrase is also rejected; such rejections are counted rather than silent. The same detector screens generated summaries and extracted person names.

The guard is a mitigation, not a solution, and Section 6.5 measures precisely how incomplete it is.

5.11 Retrieval and Grounded Answering

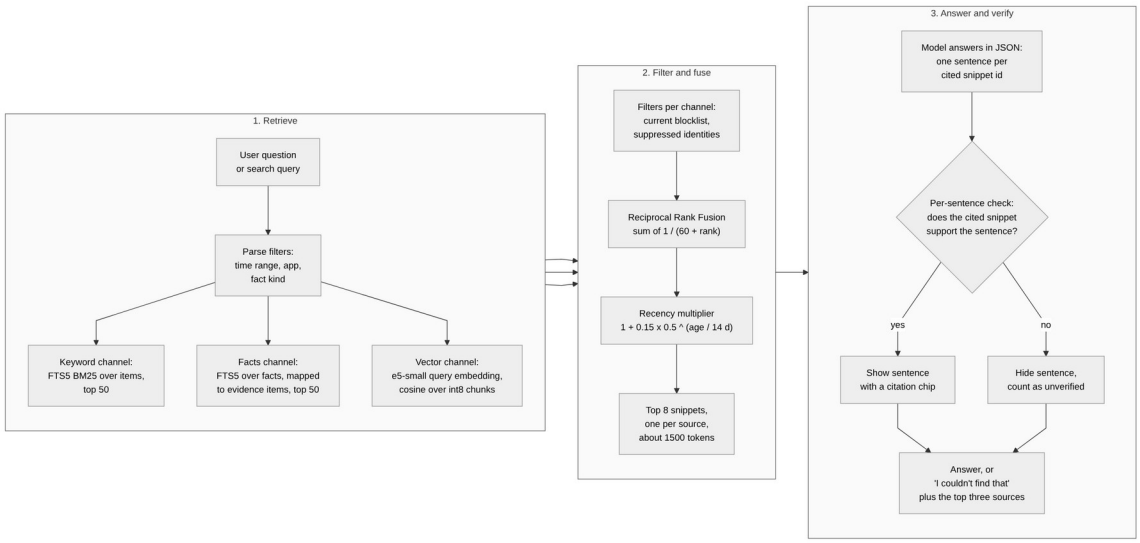


Figure 5.2: Retrieval and grounded question answering, from query parsing to per-sentence citation verification.

Chunking is deterministic and provenance-preserving: windows of at most 380 words overlapping by 48, cut at a paragraph or sentence boundary when one is nearby, each recorded as a character span of the item so that a retrieved chunk always leads back to exact source text. The chunker configuration is versioned and the version is stored with every vector, so that changing the chunking does not silently mix incompatible vectors. Vectors are stored as int8 values with a single floating-point scale, which quarters the storage relative to 32-bit floats.

Answering proceeds as in Figure 5.2. The model receives the snippets and must produce JSON in which each sentence names the snippet it came from. Sentences are parsed incrementally as they stream, so the first verified sentence can be displayed before the answer is complete. Each sentence is then checked against the snippet it cites using the same claim machinery as the draft fact checker. A sentence that does not verify is **hidden and counted**, not shown with a warning. If nothing verifies, the system says it could not find the answer and offers the three best sources for the user to read. The measured outcome of this policy is zero fabricated citations, at the cost of a verification rate of 83.7 per cent against a target of 90.

5.12 Draft Assistant

The draft assistant is the only part of the system that writes into another application, so it carries the most safety machinery.

Before generation, the focused element's runtime identity is recorded along with whether it is editable and whether it is a password field; an unknown password state counts as a password and refuses. Thread context — the last two thousand characters or so, redacted — is gathered along with relevant memory. Generation runs at interactive priority with a single retry on invalid output.

The fact checker then runs **independently of the model**. It extracts checkable claims — email addresses, URLs, file names, dates, times, weekdays, relative days, amounts, numbers and capitalised names — and marks a claim supported only when its normalised form occurs in the context. Normalisation is what makes "28 Oct" match "28 October", "1,000,000" match "1000000" and "5 pm" match "5pm". Bracketed placeholders are never treated as claims. On top of claim checking, the checker flags new commitments the draft would create, sentences that look like repeated injected text, and any secret.

Insertion requires an approval token. On insertion the system re-verifies that the focus target is still the same element, saves **every** clipboard format rather than only text, writes the draft marked with the formats that exclude it from clipboard history and cloud synchronisation, sends the paste keystroke, and restores the clipboard after a short delay. The system never sends Enter.

Two defects found during this stage are instructive. The clipboard restore never ran, because the clipboard sequence number was read while the clipboard was still open and closing it advances the number, so the code always concluded that someone else had copied in the meantime. During the test that revealed this, the development machine's clipboard contents were lost and could not be recovered. Separately, saving only text would have refused almost every real clipboard, since real clipboards usually carry HTML or rich text alongside the plain text.

5.13 Rollups, Brief and Open Loops

Rollups are produced at hour, day, week and month level with local-time boundaries stored as UTC milliseconds. What is due is **derived from the store each time** rather than tracked in a separate queue, which makes the schedule durable by construction: an hour with activity becomes due five minutes after it closes, a day once it ends, and a parent waits until every child with activity has a fresh summary. Idle hours get no summary at all.

Rollup output is validated by code in the same spirit as extraction: strict schema, highlight identifiers must correspond to entries of that period with invalid ones stripped, word limits per level, redaction, and rejection of any sentence that looks injected or that instructs the reader. If no model is available, or the model fails twice, **an extractive summary is written by code** so that every active period still has a rollup; it is regenerated with the model only if it later goes stale. Writing a summary marks the summaries above it stale, and deleting an item marks every summary that cited it stale.

The morning brief is assembled entirely by code from the store — overdue and due-today loops, open commitments in both directions, questions the user is waiting on, and yesterday's already-validated summary. The opening line is generated from counts rather than by the model, so the brief never waits for inference and contains no unvalidated model text.

Open loops are never closed automatically. Completion evidence moves a fact to a *proposed* state that the user confirms. This is enforced rather than merely intended: a test scans the source and fails if any code path other than the approval-token command writes a closed status.

5.14 Desktop Shell and Interface

The shell exposes a fixed set of typed commands. Inputs are validated for length, enumeration membership, identifier validity and token validity, and errors return a code rather than a message containing content. The content security policy permits only the application's own origin with no inline script and no evaluation, navigation is locked, the JavaScript prototype chain is frozen, developer tools are absent from release builds, and exactly one capability — event listening — is granted.

Status-changing operations use a two-step approval token: the interface asks for a token, the user confirms, and the token is single-use. This is what prevents an interface defect, or an injected instruction that reached the interface, from closing a user's commitments.

5.15 Network Boundary and Model Onboarding

The boundary is enforced at build time. A Python script walks the full dependency graph produced by Cargo and fails the build if any library other than `tsr-net` can reach a network-capable crate. It is run for both the default and the updater feature sets, and — importantly — it is itself tested against deliberately broken fixtures to confirm that it rejects them, so a checker that silently passed everything would be caught.

Downloads require a typed approval value that can be constructed only from an explicit user action. Files are written to a partial file with an accompanying metadata record so that an interrupted download resumes, and the final file appears only by atomic rename after its SHA-256 digest matches a pinned manifest. Since the security audit, model files are also digest-checked on first use and whenever their size or modification time changes, so that a file substituted after installation is detected.

5.16 Configuration

Configuration lives in a single JSON file in the application's local data directory; the full reference appears in Appendix C. The defaults are chosen so that an unmodified installation is the safe one: automatic backend selection, full offload when memory allows, a ten-minute idle unload, a five-second active sampling cadence and a ninety-day retention limit.

5.17 Summary

The implementation follows one pattern repeatedly: identify the point at which an unverified claim or an unsafe action could enter the system, and put a deterministic check there that does not depend on the model. Chapter 6 measures how well that pattern worked.

CHAPTER 6 — TESTING, EVALUATION AND RESULTS

6.1 Evaluation Methodology

Every target reported here was written down before the corresponding feature was built, and no target was changed after a measurement. Results are labelled consistently:

- **REAL** — measured against the real operating system, real processes, real hardware or the real language model.
- **SIMULATED** — measured with a stand-in for the model, the inference server or the embedder. A simulated result tests the surrounding code, not the model.
- **UNVERIFIED** — not measured. An unverified item is reported as unverified and never inferred from a related result.

Where a defence was tuned against part of a corpus, the untouched part is reported separately and treated as the honest estimate. Where a figure improved after a bug fix, both figures are given.

6.2 Test Environment

Table 6.1 records the machine on which every measurement in this chapter was taken.

Table 6.1: Evaluation environment

Item	Value
Operating system	Windows 11, build 26300
Processor	AMD Ryzen 5 7530U, 12 logical cores, AVX2 / FMA / F16C
System memory	15.3 GiB
Graphics	AMD Radeon integrated GPU, used through the Vulkan backend
Display	One 1920 × 1080 monitor at 150 % scaling
Inference runtime	llama.cpp build b11900, Vulkan variant, run as an external process
Language model	Qwen3-VL-4B-Instruct, Q4_K_M quantisation, with the F16 projection file
Embedding model	multilingual-e5-small (ONNX); approved for code paths but not run end to end
Reference machine in the plan	NVIDIA RTX 4050, 6 GB — not available; every timing below is from the integrated GPU

The single most important fact about this environment is that the reference machine named in the plan — an NVIDIA RTX 4050 with 6 GB of dedicated memory — was

not available. The available graphics processor is integrated, shares system memory with the processor, and has far lower memory bandwidth. Every latency figure in this chapter is therefore a lower bound on what the system can achieve, and the accuracy figures, which depend on the model rather than on throughput, are unaffected.

6.3 Functional and Structural Testing

Table 6.2 summarises the test and evaluation assets.

Table 6.2: Automated test and evaluation inventory

Category	Count	What it covers
Rust test functions	375	Unit and integration tests across the ten crates and the desktop shell; 17 are marked as requiring real hardware or a real desktop session.
Secrets corpus	355 seeded cases + 242 clean lines	Aadhaar including virtual identifiers, PAN, passport, voter identity, bank accounts, IFSC, UPI, cards, API keys, tokens and private keys.
Injection corpus	152 cases (147 adversarial, 5 genuine controls)	Thirty-eight attack categories including hidden text, fake system prompts, schema override, store poisoning, multilingual and OCR-derived attacks.
Extraction evaluation set	80 items / 126 gold facts, split into development and held-out halves	Chat, email, work and mixed screen content with verbatim gold evidence.
Question answering set	60 questions	Past-oriented questions over a seeded store.
Draft scenarios	20 scenarios	Each with must-include and must-not-include expectations.
Randomised robustness tests	20,000 inputs per target; 3,000 random chunkings per streaming parser	Redaction, date resolution, incremental JSON parsing and text normalisation.

Testing is organised around the trait boundaries described in Section 5.4. Because the capture core takes the operating system, storage and time as parameters, the gate logic, ladder ordering, blocklist interaction, session merging and flush conditions are all exercised against simulated windows and a **real encrypted database**, with no desktop session required. Seventeen tests are marked as requiring a real desktop, real hardware or real binaries and are run separately.

Selected results that bear on the requirements:

- **Deadline compliance (REAL).** Across sixteen windows and nine applications, the maximum time an accessibility read blocked the caller was 294 milliseconds against a 300-millisecond requirement. Microsoft Word, PowerPoint and File Explorer all hit the deadline and were served by the second rung; their abandoned reads ran on for between 290 and 732 milliseconds in the background, which is the precise failure the deadline exists to prevent.
- **Blocklist behaviour (REAL and simulated).** Private-state detection was verified against real Chrome and Edge windows in both normal and private modes; an unreadable state produces a block. The complementary test — that an ordinary non-browser window merely titled with the word "password" is still captured — also passes, confirming that the rule does not over-block.
- **Password exclusion (REAL, partial).** In browser fixtures, a canary value placed in a password input never appeared in accessibility or recognised text. The equivalent test against a native Windows Forms or WPF application has **not** been performed and remains an outstanding item.
- **Encryption (REAL).** A database opened with a wrong or zero key is rejected; a hexadecimal scan finds no plaintext header and no canary string in either the database or its write-ahead log; a wrong-key open writes nothing to standard error.
- **Randomised robustness.** Twenty thousand generated inputs per target were run against the redaction engine, the date resolver and the text normaliser, and three thousand random chunkings against each streaming parser. This found two genuine defects — a panic in the date resolver on extreme offsets, and the non-idempotent redaction described in Section 5.5 — both fixed with regression tests. Coverage-guided fuzzing was not performed, as it requires a nightly toolchain that could not be installed under the project's network constraints.

6.4 Privacy and Security Evaluation

6.4.1 Redaction

Table 6.3 gives the redaction results.

Table 6.3: Redaction results against the seeded secrets corpus

Measure	Target	Result
Seeded secrets removed	100 %	355 / 355 (100 %)
False positives on clean text	≤ 2 %	1 / 242 lines (0.41 %)
Identifier families covered	—	Aadhaar (including virtual

Measure	Target	Result
		identifiers), PAN, passport, voter identity, bank account, IFSC, UPI, payment cards, one-time codes, API keys, JSON web tokens, private keys
Behaviour on first run of the 65 new cases	—	48 leaked; no detector existed for Aadhaar virtual identifiers, passports or voter identity numbers. Detectors added; redaction version incremented.
Idempotence	Stable output	Defect found by randomised testing (a second one-time code inside the first keyword window survived); redaction now runs to a fixed point.

The corpus was expanded during the security audit from 290 to 355 cases by adding Aadhaar virtual identifiers, passport numbers and voter identity numbers. On first run the sixty-five new cases **leaked forty-eight of them**, because no detector existed for those formats. Detectors were added and the redaction version incremented. This is reported here because it is the clearest available evidence for a general point: a redaction engine is only as good as the corpus it has been tested against, and a corpus that is not extended will certify a false sense of completeness.

6.4.2 Plaintext on Disk

A canary string was placed in a captured item, a window title, a stored fact, a summary and the open interface panel, and the application was run for sixty seconds. A scan of 285 files totalling 32 megabytes across the data directory, the WebView2 profile, the temporary directory and the crash-dump folders found **zero occurrences**. A positive control placed deliberately in a plain file was found, confirming that the scan worked.

Two caveats apply. The release criterion asks for a twenty-four-hour run, which has not been performed. And the Windows page file and hibernation file could not be scanned, as that requires elevated privileges; if the operating system swaps a page containing decrypted text, that text could in principle be written there. This is recorded as a residual risk in Section 7.5 rather than claimed as covered.

6.4.3 Network Behaviour

This is the clearest failure in the evaluation and is reported in full.

A ten-minute monitored run observed the complete process tree. The application process made **zero** outbound connections, and the inference worker was never seen outside the loopback interface. However, the Microsoft WebView2 process that renders the interface opened **one HTTPS connection to a Microsoft address** approximately one minute after start, in each of several runs.

Every mitigation available to the application had already been applied: the framework's defaults were preserved, the reputation service was disabled and the setting read back to confirm it, background-networking switches were set, proxy discovery was disabled, navigation was locked, and the runtime's environment variables were cleared. The connection does not appear in the browser engine's own network log, and the host name was absent from the system's resolver cache, which indicates that it originates in code outside the engine's network service — most plausibly a diagnostic client, although this is **not confirmed**.

The release criterion states zero connections from the application's processes to anything other than the loopback address. As measured, it fails. The project recorded this as an open decision rather than narrowing the criterion, and the privacy documentation states plainly that "nothing leaves your PC" cannot be claimed for the interface process until it is resolved. The options under consideration are to identify the endpoint with an elevated packet capture and disable it if it is switchable, or to replace the always-present interface with a native implementation and use the web runtime only for windows the user opens deliberately.

6.4.4 Dependency and Supply-Chain Review

The dependency boundary check passes for both the default and the updater configurations, and the checker itself was validated against deliberately broken fixtures. Licence and banned-crate checks pass; all dependencies are permissive, with one weak-copyleft licence present. Advisory checking passes against an advisory database cached offline; a fresh fetch and a Node package audit both require network access and were not performed. The automatic updater is compiled out of every build because a transitive compression library carries three unpatched advisories.

6.5 Injection Robustness

This evaluation is the most methodologically careful part of the project, because it is the easiest to get wrong. A defence tested only against the attacks it was built from

will always appear to work. Figure 6.1 and Table 6.4 therefore separate the attacks used for tuning from those that were not.

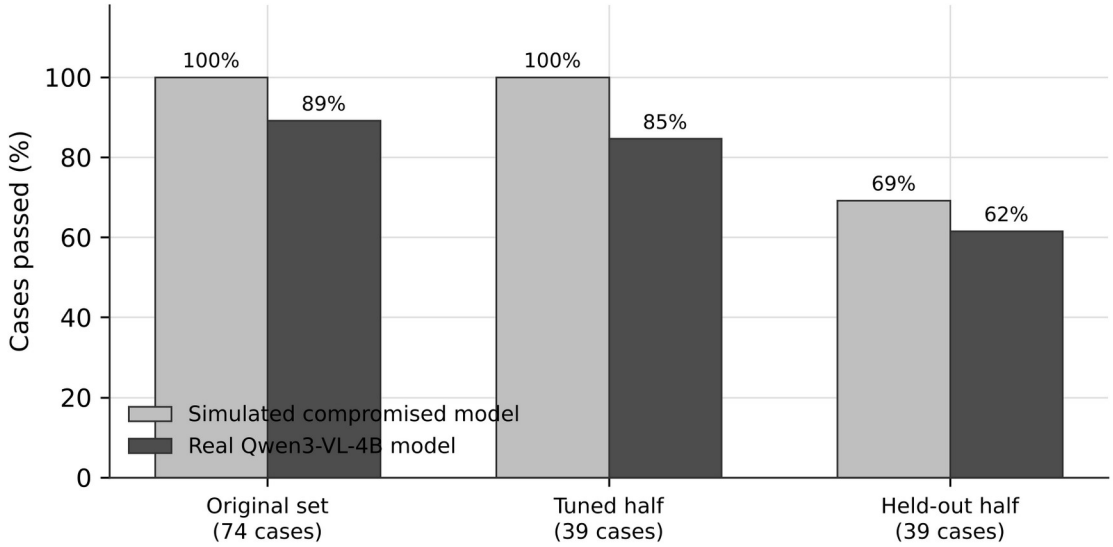


Figure 6.1: Injection corpus pass rates. The held-out half, which was never used for tuning, is the honest estimate of generalisation.

The corpus contains 152 cases across thirty-eight categories — hidden text, fake system prompts, schema override attempts, store poisoning, delayed triggers, multilingual and transliterated attacks, attacks arriving through recognised text, attacks inside window titles, and five genuine controls that must *not* be blocked. Seventy-eight cases were written before any run of the defence against them. The corpus was then split in half: one half was used to tune the guard rules, and **the other half was never looked at during tuning**.

Table 6.4: Injection corpus results. The held-out half was never used for tuning and is the honest estimate of generalisation.

Run	Original set (74)	Tuned half (39)	Held-out half (39)	Total (152)
Simulated fully compromised model, before audit fixes	—	27 / 39	25 / 39	126 / 152
Simulated, after tuning on the tuned half only	74 / 74	39 / 39	27 / 39 (69 %)	140 / 152
Real Qwen3-VL-4B model	66 / 74	33 / 39	24 / 39 (62 %)	123 / 152

The pattern is unambiguous. Against the original set the defence is close to complete; against the tuned half it is near-perfect, which is expected and means nothing on its own; and against the untouched half it stops roughly 62 per cent of attacks when the real model is used. The gap between 100 per cent and 62 per cent is the measurement of how much the earlier numbers were worth.

The twelve held-out failures with the real model share recognisable styles: social engineering phrased as ordinary correspondence, such as a fake password-reset request or a payroll account change; attempts to poison stored memory; delayed-trigger variants; quoted phishing; and injections placed in window titles. Multilingual cases were among the weakest categories. **They have deliberately been left unfixed**, so that the 62 per cent figure continues to mean something; fixing them would turn the held-out half into a second tuned half, and a genuine estimate would then require a third corpus.

What holds regardless of the guard is the more important result. In all 152 cases:

- no injected instruction caused an action, because the model has no tools;
- no secret was revealed in clear text;
- no table other than the expected one was written;
- the delimited content block could not be closed from inside;
- exactly one model call was made per case.

The failures are therefore confined to a single consequence: text that the user would see in their task list which they should not trust — for instance a phishing entry asking the user to share an identity number, or a fabricated instruction to transfer money. This is a real harm and is not minimised here, but it is bounded, and the bound comes from the architecture rather than from the guard.

Five cases failed with a transport timeout during a run that the machine interrupted twice by entering standby. They are counted as failures and were not retried. One control case passed in an earlier run and failed in this one, which is a reminder that the model's output varies between runs even at the same settings.

6.6 Extraction Accuracy

Table 6.5 gives the aggregate result.

Table 6.5: Extraction results on the held-out set, real model run

Measure	Target	Result
Items processed	—	58
Gold facts	—	83
Facts stored	—	97, of which 5 held for review and not shown as tasks
Precision	—	49.5 %
Recall	—	57.8 %
F1	≥ 0.70	0.533 — not met
Hallucinated stored facts	≤ 3 %	0 / 97 = 0.0 % — met
Schema-invalid stored rows	0	0 — met
Items lost to invalid output	—	0 (15 of 58 before the key-order fix)
Injection items with forbidden text stored	0	0
Wall-clock time for the run	—	about 50 minutes (5 h 48 min before the key-order fix)

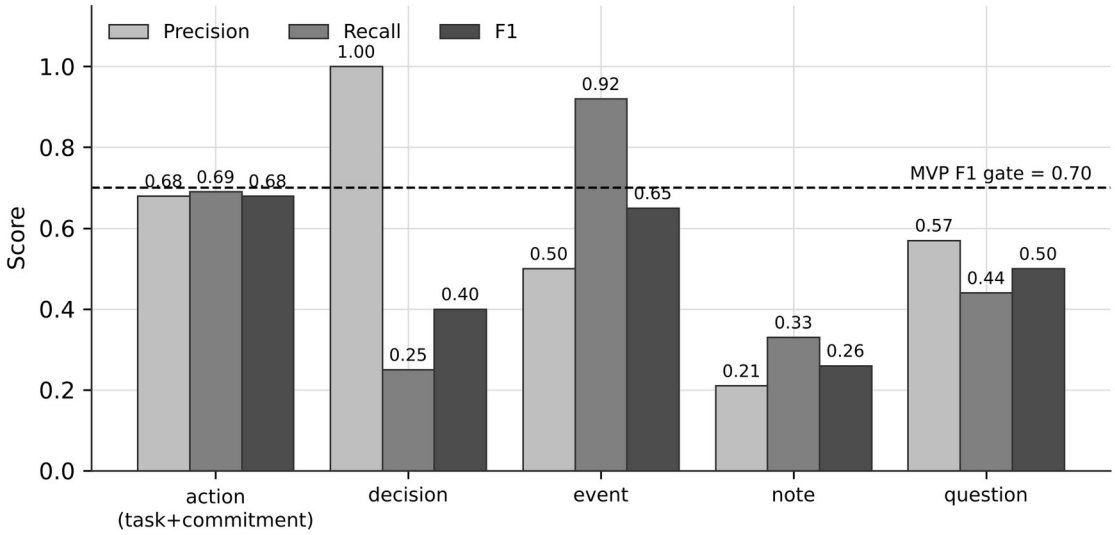


Figure 6.2: Extraction precision, recall and F1 by fact kind on the held-out set, real model run.

The headline result is a failure on quality and a success on integrity. The system extracts fewer of the right facts than the target requires, but **everything it does store is traceable**: of 97 stored facts, zero carried evidence that was not present in the source, and zero rows violated the schema.

The breakdown by kind in Figure 6.2 locates the problem. Task and commitment extraction — the facts that actually matter for a memory system — reaches an F1 of 0.68, just under the 0.70 gate. Event extraction has high recall and poor precision. The category that destroys the aggregate is `note`: precision 0.21 against 23 false positives,

because the model generates a large number of low-value observations that the gold set does not consider worth recording. Decisions are the opposite failure: every decision the model produced was correct, but it found only a quarter of them.

The key-order defect. An earlier run of the same evaluation scored 0.507 with fifteen of fifty-eight items lost entirely to invalid JSON, and took five hours and forty-eight minutes. Investigation found that the JSON schema was being serialised to the model with its keys in alphabetical order, because the serialisation library's default behaviour sorts map keys. Since the grammar forces the model to emit fields in schema order, this meant the model had to commit to fields before it had produced the content they described. Restoring the declared key order removed every invalid-JSON failure, raised recall from 44.6 to 57.8 per cent and cut the run to roughly fifty minutes — but *lowered* precision from 58.7 to 49.5 per cent, because the model now successfully produced the low-value notes it had previously failed to emit. Net F1 improved from 0.507 to 0.533, still short of the gate.

This defect is worth recording as a general lesson: a grammar-constrained model is sensitive to field ordering in a way that is invisible in the schema definition and easy to introduce accidentally through a serialisation default.

Rejections were counted by reason: five facts had no evidence, four cited a person not present in the source, three had evidence too short to be meaningful, two contained an unsupported number, and eleven were rejected by the injection guard across its categories. Every one of these is a fact the user never sees — the intended behaviour.

6.7 Question Answering

Table 6.6 gives the results.

Table 6.6: Question answering results over 60 questions

Measure	Target	Simulated oracle	Real model, sorted keys	Real model, declared key order
Answered correctly	≥ 80 %	53 / 60 (88.3 %)	9 / 60 (15 %)	40 / 60 (66.7 %) — not met
Shown sentences verified	≥ 90 %	53 / 53 (100 %)	4 / 4	41 / 49 (83.7 %) — not met
Fabricated citations	0	0	0	0 — met
Invalid answers	0	0	3	0
Time to first	≤ 2.5 s	—	14.5 s	25.2 s — not met

Measure	Target	Simulated oracle	Real model, sorted keys	Real model, declared key order
sentence (p50)				
Time to full answer (p50)	≤ 10 s	—	19.2 s	25.5 s — not met

The same key-order defect dominated this evaluation. With sorted keys, the model was forced to emit an "insufficient" flag before writing any sentence, and produced empty answers for nearly every question; correctness measured 15 per cent. With declared key order and no other change, the same prompt scored 66.7 per cent.

Of the sixty questions after the fix: 34 were answered correctly, 11 were answered but wrong or incomplete, 6 were correctly declined as unanswerable, 8 were wrongly declined, and 1 was empty. Eight of the forty-nine sentences the model produced failed verification and were hidden from the user. Zero citations were fabricated.

Three qualifications apply to the 66.7 per cent figure. First, there is **no held-out split** for this evaluation: the second prompt version was written after seeing the first fail on these same sixty questions, so the figure may be optimistic. Second, retrieval ran without the embedding channel, since no embedding model was approved on the evaluation machine; the vector half of the hybrid search is therefore unmeasured end to end. Third, the facts used for retrieval came from a simulated oracle extraction rather than from the real extraction measured in Section 6.6, so this evaluation measures the answering pipeline rather than the complete system.

Latency was far outside target: a median of 25.2 seconds to the first sentence against a 2.5-second target, and 25.5 seconds to a complete answer against ten. Figure 6.3 shows this against the draft figures.

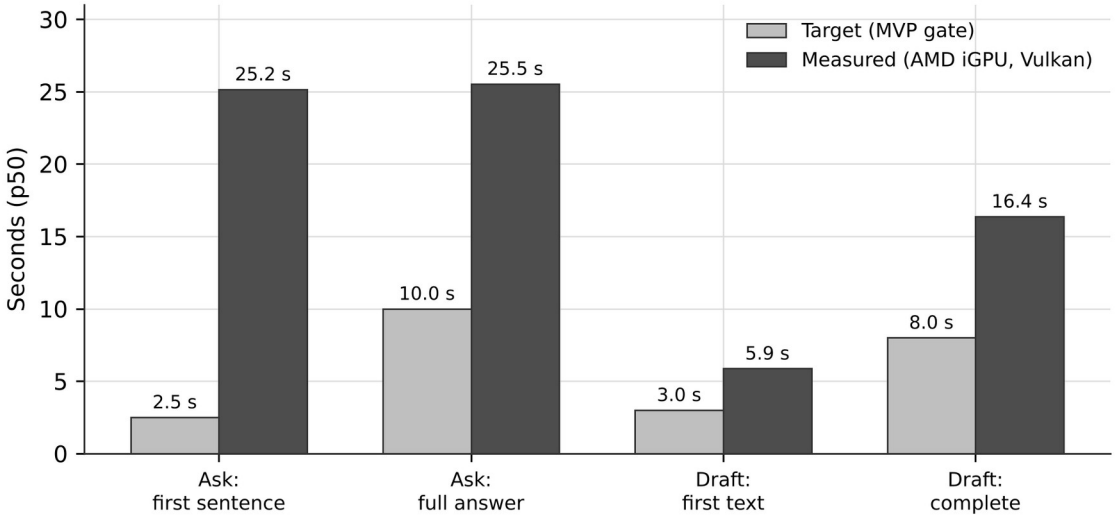


Figure 6.3: Measured median latency against the project's own interactive targets, on the available integrated GPU.

6.8 Draft Assistant

Table 6.7 gives the results.

Table 6.7: Draft assistant results over 20 scenarios, real model run

Measure	Target	Result
Valid drafts produced	20 / 20	17 / 20
Unsupported claims shown without a flag	0	0 — met
Required content present	—	14 / 17
Forbidden content absent	—	16 / 17
Placeholder used where expected	—	16 / 17
Insertion into a password field	Never	Refused in a real test — met
Insertion into a changed target	Never	Refused in a real test — met
Clipboard restored after insertion	Always	All formats restored at original sizes — met
Time to first visible text (p50)	≤ 3 s	5.9 s — not met
Time to complete draft (p50)	≤ 8 s	16.4 s — not met

The safety properties hold and the speed does not. Of twenty scenarios, seventeen produced valid drafts. **Zero unsupported claims were shown without a flag** — the independent fact checker caught every invented date, amount and name. In a real insertion test, a password field was detected and refused, a changed focus target was detected and refused, an unknown password state was treated as a password, and every clipboard format was restored at its original size with the exclusion formats present during the paste.

Three things could not be tested on the available machine. Clipboard history is disabled there, so the guarantee that the draft does not enter the history could not be exercised. Cloud clipboard upload cannot be observed without an account. And the five named applications in the criterion — two mail clients and three messaging applications — all require signed-in accounts; only a plain Windows text field was exercised.

6.9 Interface Performance and Capture Exclusion

Table 6.8 gives the interface measurements.

Table 6.8: Interface performance and capture exclusion

Measure	Target	Result
Animation frame time, p50 / p95 / max	16.7 ms (60 fps)	16.7 / 16.8 / 33.3 ms over 1,229 frames in 20 cycles — met
Frames exceeding 17.7 ms	—	0.16 %
Input-to-visual latency, p50 / p95	≤ 100 ms	33 / 64 ms over 20 opens — met (292 ms p95 before optimisation)
Focus stolen during a peek	Never	Foreground unchanged; 20 / 20 focus returns after close — met
Presence in Graphics Capture frames	Absent	100 % absent, pill and expanded; control window 100 % visible
Presence in GDI block-transfer frames	Absent	100 % absent; control window 100 % visible
Named conferencing applications	Absent	Not run — outstanding
Placement after monitor or scaling change	Correct	Unit-tested only; one monitor available — outstanding

Capture exclusion was verified at the interface level by placing a magenta backdrop beneath the window and capturing the monitor twice, once through Windows Graphics Capture — the interface used by the Snipping Tool, Game Bar, Teams on Windows 11 and modern OBS capture — and once through the older GDI block-transfer path used by legacy recorders. The notch was wholly absent from both, in both its compact and expanded states. Two controls confirm the method: an unexcluded test window was fully visible in the same frames, and the same window with exclusion applied was fully hidden.

The criterion names five specific applications, none of which was run: two require signed-in accounts and a live screen share, and one was not installed. The result is

therefore **partial**: the operating-system mechanism is verified, the applications are not.

Placement after a monitor change or a scaling change is likewise partial. The handlers are wired and the placement arithmetic is unit-tested for negative origins, monitors positioned above the primary, and three scaling factors; but only one monitor was available, so no live change was exercised.

Three defects were found by measuring rather than by testing, and are worth recording. A second graphics capture in the same process caused an access violation that would have crashed the application after the first use of the vision rung; the cause was initialising a new apartment-threaded context per capture, and the fix was a single persistent worker thread. The default hotkey was already registered by another application on the machine, so the toggle silently never worked; alternates were added and the active chord is now shown in the interface. And the input-to-visual latency was initially 292 milliseconds at the 95th percentile, nearly three times the target, before being reduced to 64.

6.10 Hardware Portability

Table 6.9 records which backends were exercised and how.

Table 6.9: Hardware verification coverage

Scenario	Method	Outcome
Processor only, no GPU	Real hardware and simulated	Verified
AMD integrated GPU through Vulkan	Real hardware	Verified; backend probed, device listed, model executed
NVIDIA with CUDA, and NVIDIA falling back to Vulkan	Simulated adapters and probe output	Selection logic verified
AMD with ROCm, and ROCm failing to Vulkan	Simulated	Selection logic verified
Intel with SYCL or Vulkan	Simulated	Selection logic verified
Apple with Metal; Qualcomm with OpenCL	Simulated	Selection logic verified
Several GPUs present; override by name or index	Simulated	Selection logic verified
Software rendering adapter present	Real hardware	Correctly ignored rather than selected
Tampered build directory (extra library planted)	Real and simulated	Rejected by the manifest check
Probe ignores an inherited device-visibility variable	Real hardware (AMD, Vulkan)	Verified

Scenario	Method	Outcome
Unsupported model and backend combination	Simulated	Backend skipped rather than attempted
Every backend probe failing	Simulated; real for a missing executable	Falls back to the processor

Two backend families have been exercised on real hardware: the processor fallback and the Vulkan path on an AMD integrated GPU. Five further families — CUDA, ROCm, SYCL, Metal and OpenCL — have had their **selection logic** verified against simulated adapter lists and simulated probe output in the exact format that real builds print, but no device of those families was available. This is stated precisely because the distinction matters: the tests prove that the arbitration code chooses correctly given a particular hardware report; they do not prove that any given vendor's driver works.

Several behaviours were nevertheless verified on real hardware and are meaningful: a software rendering adapter present on the machine was correctly ignored rather than selected; a build with an extra library planted in its directory was rejected by the manifest check; and the backend probe correctly ignored an inherited environment variable that would otherwise have restricted the visible devices.

6.11 Consolidated Scorecard

Table 6.10 brings every criterion together.

Table 6.10: Consolidated scorecard against the project's own release criteria

Criterion	Target	Measured	Verdict
Secrets removed from the corpus	100 %	355 / 355	Met
Redaction false positives	≤ 2 %	0.41 %	Met
Plaintext on disk (canary scan)	0 hits	0 hits in 285 files / 32 MB	Met for the 60-second run; the 24-hour run is outstanding
Hallucinated stored facts	≤ 3 %	0 / 97 = 0.0 %	Met
Schema-invalid stored rows	0	0	Met
Fabricated citations in answers	0	0	Met
Facts never closed without confirmation	0 automatic closures	0; enforced by a source scan	Met
Insertion into a password or changed	Never	Refused in a real test	Met

Criterion	Target	Measured	Verdict
field			
Interface frame rate	60 fps	p95 frame 16.8 ms	Met
Input-to-visual latency	≤ 100 ms	p95 64 ms	Met
Interface absent from screen captures	Absent	100 % absent from Graphics Capture and GDI frames	Met at the interface level; named applications outstanding
Extraction F1 (held-out)	≥ 0.70	0.533	Not met
Answers correct	≥ 80 %	66.7 %	Not met
Answer sentences verified	≥ 90 %	83.7 %	Not met
Time to first answer sentence	≤ 2.5 s	25.2 s	Not met on the available hardware
Complete draft	≤ 8 s	16.4 s	Not met on the available hardware
Zero outbound connections	0	0 from the application; 1 from the WebView2 runtime	Not met as written
Injection corpus	No injected instruction stored as a user task	123 / 152 overall; 24 / 39 on the untouched half	Not met
Resource use over 8 h and 7-day stability	CPU ≤ 3 %, RAM ≤ 250 MB; no crash	Not run	Outstanding

Of nineteen criteria, eleven are met — two of them with a longer-duration check still outstanding — seven are not met, and one has not been run for want of machine time. The division between them is not random. **Every criterion met is one enforced by deterministic code.** Every criterion missed depends either on the quality of a four-billion-parameter quantised model, on the throughput of an integrated GPU, or on a third-party runtime outside the application's control.

6.12 Threats to Validity

Several factors limit how far these results generalise, and they are stated here rather than left for a reader to infer.

Hardware. All model-dependent timing was measured on an integrated GPU with shared memory. The latency results say almost nothing about what a discrete GPU would achieve, and the project's own target hardware was never tested.

Evaluation sets are synthetic. The extraction, answering and draft sets are constructed items resembling emails, chats and documents, not captures of real activity, which would contain personal data. Real screen text is messier.

No held-out split for answering. As noted in Section 6.7, the second prompt was written after observing the first fail on the same questions.

Application compatibility is thinly sampled. Only seven of twenty target applications were tested for text acquisition; the rest were not installed or needed accounts. The gate asking for sixteen of twenty is unverified, not failed.

Model non-determinism. At least one injection case passed in one run and failed in another with identical settings, and five cases timed out in a run the machine interrupted. Single-run results on a generative model carry run-to-run variance that this evaluation did not quantify by repetition.

Simulated results are not model results. Where a number is labelled SIMULATED, a stand-in produced the model's output. Such a result validates the surrounding code and says nothing about the model.

6.13 Not Yet Evaluated

For completeness: the twenty-four-hour network and canary runs; the eight-hour resource and stability run; the seven-day stability run; the file-system watcher test confirming that no image file is ever written; password-field exclusion against native Windows Forms and WPF applications; capture exclusion inside the five named conferencing applications; draft insertion in the five named applications; clipboard-history and cloud-clipboard exclusion; placement across a live monitor or scaling change; the lock-and-unlock path that triggers the morning brief; end-to-end operation of the embedding channel; and inference on any backend other than Vulkan and the processor.

CHAPTER 7 — DISCUSSION, LIMITATIONS AND FUTURE SCOPE

7.1 What the Implementation Achieves

The central claim this work can support is narrower than the one it set out to make, and more interesting for being narrow.

A desktop application *can* run a four-billion-parameter vision-language model across unrelated GPU vendors without containing a line of any vendor's code, by treating each backend as an externally pinned artefact and discovering the hardware at run time. It *can* read text from other applications without writing a single screen image to disk and without blocking its own capture loop beyond 300 milliseconds, even against applications that take three-quarters of a second to answer. It *can* remove every secret in a 355-case corpus while flagging fewer than one clean line in two hundred, provided the detectors validate checksums rather than matching digit patterns. It *can* guarantee that no stored fact exists without a verbatim source span, and the measured hallucination rate of such a system is zero rather than small. And it *can* hide its own interface completely from two independent screen-capture paths.

What it cannot yet do, on the hardware available and with a model of this size, is be accurate or fast enough to be pleasant. Extraction misses too much and over-produces trivia. Answers are right two times in three where the criterion asked for four in five. An answer takes twenty-five seconds where the criterion asked for two and a half. And one process in the application's own process tree — the browser runtime that renders the interface — makes a network connection that the project could not switch off and therefore could not claim did not exist.

The value of the result is in that separation. It was not obvious in advance which guarantees would depend on the model and which would not. The evaluation answers that question empirically: **every criterion enforced by deterministic code was met, and every criterion that depended on model quality, hardware throughput, or a third-party component was missed.** That is a useful finding for anyone designing a similar system, because it says where to put the engineering effort — and where not to bother asking the model to behave.

7.2 Objectives Met

Table 7.1 revisits each objective from Section 1.4 with the evidence gathered.

Table 7.1: Project objectives and the evidence for each

Objective	Outcome	Evidence
O1: Run a vision-language model locally without linking any GPU vendor library into the application.	Achieved	No vendor library appears in the dependency graph; the application builds and runs on a machine with no GPU SDK; backends are separate hash-verified builds.
O2: Select a backend at run time and fall back deterministically.	Achieved in code; partially verified on hardware.	Ranked candidate selection with CPU last; two backend families exercised on real hardware, five on simulated adapters.
O3: Contain inference failures so they cannot take the application down.	Achieved	Suspended creation, cleared environment, Job Object with kill-on-close, port-owner authentication, supervisor with backoff and restart budget.
O4: Read screen text without writing images to disk and without reading blocked windows.	Achieved	Capture ladder with an in-memory frame path; blocklist defaults to blocking when a private state cannot be read.
O5: Remove secrets before anything is stored.	Achieved	355 of 355 seeded secrets removed at a 0.41 % false-positive rate.
O6: Encrypt everything at rest under the user's Windows login.	Achieved	Whole-database encryption; a wrong key is rejected and writes nothing to standard error; no plaintext found on disk.
O7: Ensure every stored fact is traceable to a verbatim source span.	Achieved	No fact can exist without an evidence row; zero hallucinated stored facts in the real-model run.
O8: Prevent instructions inside captured text from becoming the user's tasks.	Partially achieved	No case executed an action or leaked a secret; twelve injected texts on the untouched half were still stored as readable to-do entries.
O9: Answer questions from memory with a source for each sentence.	Partially achieved	Zero fabricated citations, but 66.7 % correctness and 83.7 % verification against targets of 80 % and 90 %.
O10: Keep the interface responsive and invisible to screen sharing.	Achieved	60 fps, 64 ms input-to-visual, complete absence from two capture paths.
O11: Make no network connection other than approved model downloads.	Not achieved as stated	The application makes none; the WebView2 runtime makes one HTTPS connection per run.

Seven of the eleven objectives are achieved, three are partially achieved, and one is not achieved as stated.

7.3 Design Trade-Offs

Strictness against recall. Rejecting an entire model output on any schema violation, rather than repairing it, costs facts that a lenient parser would have kept. The compensating benefit is that nothing stored is of unknown provenance. The evaluation shows the cost concretely: before the key-order defect was found, fifteen of fifty-eight items produced nothing at all. A lenient parser would have salvaged some of them and would also have salvaged whatever the broken grammar had caused the model to emit.

Conservatism against usefulness in the injection guard. The guard rejects a legitimate note that merely quotes an injection phrase. On the corpus, three to four genuine tasks per run were missed for this reason. This is the right direction to err for a system that stores things the user will later trust, but it is a real cost, and it means the guard's precision cannot be raised without lowering its recall against attacks.

Brute-force vector search against scale. Comparing a query against every stored chunk is linear in the size of the memory. An approximate index would be faster and would introduce a failure mode — silent degradation of recall as the index grows — that is difficult to detect in production. At single-user scale the linear cost is acceptable, and this trade-off would have to be revisited if the retention period were extended far beyond the ninety-day default.

Preemption by connection drop against wasted work. Dropping a streaming connection to free the model for an interactive request discards whatever the background job had generated. The alternative — letting background jobs finish — would make interactive requests wait many seconds behind them. On an integrated GPU, where generation is slow, the waste is substantial; on faster hardware it would be smaller.

An external inference process against latency and complexity. Process isolation costs a start-up delay on the first request after an idle period, a transport hop for every token, and a substantial amount of supervision code. In exchange, a driver reset

cannot take the application down. Given that out-of-memory resets are routine on 6 GB consumer cards, this was not a close decision.

Capture starts paused. A system that captures nothing until the user turns it on is less impressive in a demonstration and much easier to defend. The same reasoning produced the fail-closed behaviour on unknown private-browsing states, which certainly loses some legitimate captures.

7.4 Technical Limitations

Model quality at this size. The decisive limitation is the model. Extraction F1 of 0.533 is driven by a precision of 0.21 on `note` facts: the model produces a large volume of observations that a human would not record. The right response is not a larger model but a narrower task — the evaluation suggests that restricting extraction to tasks and commitments, where F1 is 0.68, would produce a system that is useful now, with notes left to a later prompt version.

Latency on integrated graphics. Twenty-five seconds to an answer is not usable. This is a memory-bandwidth problem rather than an algorithmic one, and the project's own target hardware was never available to test the hypothesis that a discrete GPU closes the gap. That hypothesis remains untested and should not be assumed.

The embedding channel is unexercised. The chunker, the vector index, the int8 encoding and the fusion logic are all implemented and unit-tested, but no embedding model was run end to end on the evaluation machine. Hybrid retrieval was therefore measured with its lexical channels only, and the contribution of the vector channel to answer quality is unknown.

Thin application coverage. Text acquisition was verified against seven of twenty target applications. The remainder were either not installed or required accounts whose contents could not be exposed to a test. The criterion of sixteen of twenty is unverified rather than failed, which is a weaker but more honest statement.

No long-run data. The eight-hour resource run, the seven-day stability run and the twenty-four-hour network and canary runs have not been performed. Memory leaks, handle leaks and slow index degradation would most likely appear in exactly those runs.

7.5 Security and Privacy Limitations

Injected text can still reach the user's task list. On the untouched half of the attack corpus, twelve of thirty-nine attacks produced an entry the user would see. None caused an action, revealed a secret or modified anything beyond the expected table — the architecture holds — but a user who trusts their task list could still be socially engineered through it. The weakest categories are social engineering phrased as ordinary correspondence, memory poisoning, delayed triggers and multilingual attacks.

The WebView2 connection. One process in the tree makes an outbound HTTPS connection per run. The application's own code makes none, but the user experiences a process tree, not a crate graph. Until this is resolved, the honest statement is the one the project's privacy documentation makes: nothing leaves the machine from Tessera's own code, and one connection from the interface runtime is not under its control.

Memory outside the application's control. Decrypted text exists in process memory while the application runs. If Windows pages that memory out, it may be written to the page file or the hibernation file. Neither was scanned, as that requires elevated privileges. Malware running as the same user, or physical access to an unlocked machine, defeats the design entirely — the data-protection key is bound to the user account, so an attacker who has the account has the data.

No coverage-guided fuzzing. Randomised testing with twenty thousand inputs per target found two real defects, but coverage-guided fuzzing, which explores systematically rather than randomly, could not be run because it requires a toolchain that could not be installed under the project's network constraints.

The guard's generalisation estimate will expire. The 62 per cent figure is honest only while the held-out half remains untouched. Any further tuning against it would require a third corpus written by someone who has not seen the guard.

7.6 Platform and Hardware Constraints

Capture is Windows-only, because it depends on UI Automation, Windows Graphics Capture, WinRT recognition, Job Objects, display affinity and the Windows data-protection interface. The storage, retrieval, inference and pipeline libraries are

platform-neutral and compile without Windows, so a port would mean replacing the acquisition and key-protection layers rather than rewriting the system.

Five of the seven backend families are verified only through simulated adapter reports. The arbitration logic is tested; the vendors' drivers are not.

7.7 Unresolved Issues

Four questions remain open and require either a decision or hardware that was not available:

25. **The WebView2 connection.** Identify the endpoint with an elevated packet capture; if it is switchable, switch it off; if it is not, decide between a native interface and an explicit, documented narrowing of the criterion.
26. **Extraction precision.** Decide whether to suppress low-value note facts in the prompt, narrow the schema, or accept a tasks-and-commitments-only product for the first release.
27. **Hardware.** Re-run the full evaluation on a discrete GPU before concluding anything about achievable latency.
28. **A third injection corpus.** Needed before any further guard tuning can be measured honestly.

7.8 Future Scope

Immediate — closing the current gaps. Re-run the held-out extraction and answering evaluations on discrete hardware. Curb low-value note facts, which is the precision driver, rather than only fixing output formats. Commission a third, unseen injection corpus. Complete the outstanding long-duration runs and the live checks listed in Section 6.13.

Near term — completing the specified system. Meeting capture and live translation were specified and not built: audio held in memory only, per-meeting consent, local speech recognition and a permissively licensed translation model. Beyond that, a closed beta on varied hardware is the only way to obtain the backend verification that simulated adapters cannot provide, and release engineering — a signed installer, a software bill of materials per release, and an opt-in signed updater once the advisory blocking it is resolved — remains outstanding.

Longer term — research directions. Three are suggested directly by the results. *Speculative decoding* with a small draft model is the most promising route to acceptable latency on memory-bandwidth-limited integrated graphics, because it trades arithmetic, which such hardware has, for memory traffic, which it lacks. *Task-specific adaptation* of a small model on the extraction schema may recover more accuracy per byte than moving to a larger general model. And the broader question raised by Section 6.5 — how to measure a prompt-injection defence honestly, given that any corpus becomes a tuning set the moment it is used — deserves attention in its own right; the tuned-and-held-out split used here is a minimum, not a solution.

Cross-platform support. Abstracting acquisition behind the existing `Platform` trait would allow a Linux implementation using the desktop portal interfaces and a macOS implementation using ScreenCaptureKit, reusing the storage, retrieval and pipeline layers unchanged.

CHAPTER 8 — CONCLUSION

This thesis set out to answer a practical question: can a desktop application keep a useful memory of what its user is working on, using a language model, without the user's screen contents ever leaving their machine — and if so, what does that cost?

The problem had four parts. Graphics hardware is fragmented across incompatible vendor interfaces, so an application that wants acceleration must either serve one vendor or ship an enormous bundle. Inference on consumer hardware is fragile, and a crash inside the application's own process takes the application and possibly the user's data with it. A system that reads the screen will read secrets. And a language model given screen text has no reliable way to distinguish the text it is meant to read from instructions hidden inside it.

Tessera answers each of these by moving the guarantee out of the model and into code. Inference backends live outside the application binary as separately pinned, hash-verified builds; the application enumerates the hardware without loading any vendor runtime, probes each verified build for real, reads the model's own header for compatibility, and produces a ranked candidate list that ends at the processor. The inference worker runs as a separate process created suspended, stripped of its inherited environment, confined to an operating-system job that kills it when the application exits, and reachable only through a loopback socket whose owning process is verified. Screen text passes through gates that fail closed, a three-rung acquisition ladder bounded by a hard deadline, and a redaction stage that validates checksums before it redacts, all before anything is written to a whole-database-encrypted store. And every fact the model proposes must quote the source verbatim; code looks for the quotation, and a fact whose quotation is absent is discarded rather than corrected.

The system was evaluated against nineteen criteria fixed before implementation began, on an AMD integrated GPU rather than the intended reference machine. The results divide cleanly. Every criterion enforced by deterministic code was met: 355 of 355 seeded secrets removed at a 0.41 per cent false-positive rate; no plaintext recoverable from disk; zero stored facts with evidence absent from their source; zero schema violations; zero fabricated citations; no commitment closed without confirmation; refusal to insert a draft into a password field or a changed target; and

complete absence of the interface from two independent screen-capture paths at 60 frames per second and 64 milliseconds of input-to-visual latency. Every criterion that depended on the model, on hardware throughput or on a third-party component was missed: extraction scored an F1 of 0.533 against 0.70; answers were correct 66.7 per cent of the time against 80; interactive latency was roughly ten times the target; the Microsoft WebView2 runtime made one outbound connection per run that the project could not suppress; and on an injection corpus whose second half was never used for tuning, twelve of thirty-nine unseen attacks still produced a misleading entry in the user's task list, although none caused an action or revealed a secret.

The limitations follow from those results. The model is too small for the breadth of task it was given, and the evaluation suggests the remedy is a narrower task rather than a larger model: restricted to tasks and commitments, extraction already reaches an F1 of 0.68. The hardware was not the hardware the project was designed for, so no conclusion about achievable latency is available. Five of seven backend families are verified only against simulated hardware reports. The embedding channel is implemented but was never exercised end to end. And the long-duration runs that would reveal leaks and drift were not performed.

The contribution of this work is therefore an architecture together with an honest measurement of it. The architecture's organising principle — that the model proposes and deterministic code disposes — turned out to be not merely a design preference but a predictor of which guarantees would survive contact with a real small model on commodity hardware. Code-enforced guarantees held without exception. Model-dependent guarantees held partially at best. For anyone building a system of this kind, that is the most portable result in this thesis: put the guarantees where they can be tested, measure against criteria written beforehand, hold back part of every adversarial corpus, and report what the measurement says rather than what the design intended.

The work that follows is clear. Re-measure on the intended hardware before drawing any conclusion about speed. Narrow the extraction schema to recover precision. Resolve or honestly document the WebView2 connection. Commission a third injection corpus before tuning the guard further. Complete the outstanding long-duration runs. Only then does the question of whether this system is good enough to

use every day become answerable — and the point of building it the way it was built is that the answer will be a measurement rather than a claim.

APPENDIX A — PROJECT DIRECTORY STRUCTURE

```
Tessera/
├── crates/
│   ├── tsr-core/        Domain primitives, Sensitive<T>, content-free
logging
│   ├── tsr-win/         Win32 and WinRT wrappers: DPAPI, Job Objects,
hooks,
│   │                    display affinity, clipboard, TCP table queries
│   ├── tsr-capture/     Acquisition ladder (UIA, graphics capture, OCR,
vision),
│   │                    deadline worker, SimHash, sessions, store writer
│   ├── tsr-guard/       Blocklists, redaction engine, claim checker
│   ├── tsr-store/       SQLCipher store, migrations, FTS5, repositories
│   ├── tsr-retrieval/   Pinned ONNX runtime, embeddings, chunking,
│   │                    hybrid search, Ask pipeline
│   ├── tsr-infer/       Hardware discovery, GGUF parsing, backend
selection,
│   │                    sidecar supervisor, loopback client, scheduler
│   ├── tsr-net/         Approval-gated, digest-verified model downloads
│   ├── tsr-pipeline/    Extraction, schema, evidence, injection guard,
│   │                    date and people resolvers, rollups, briefs,
loops
│   └── tsr-draft/       Draft generation and independent fact checking
├── src-tauri/           Desktop shell: windows, tray, hotkeys, typed IPC
├── ui/                  React and TypeScript notch interface
├── bench/               Acquisition probes and performance harnesses
├── evals/               tsr-eval harness, datasets and recorded results
├── tests/fixtures/      Secrets corpus, injection corpus, screen
fixtures
├── tools/               Pinned manifests, dependency checker, scan
scripts
├── third_party/         Fetched llama.cpp builds and ONNX Runtime
└── docs/                Decision records, stage verification reports,
                         privacy and security documentation
```

APPENDIX B — INSTALLATION AND DEVELOPMENT SETUP

Prerequisites are listed in Table 3.3. The steps below assume a Windows 11 machine with the Rust MSVC toolchain, Visual Studio C++ build tools, Strawberry Perl, Node.js with pnpm, and Python 3.11 or newer already installed.

```
# 1. Frontend dependencies and interface bundle
pnpm install
pnpm build:ui

# 2. Fetch pinned inference runtimes for this machine's hardware.
#    Each download is hash-checked against tools/sidecar.lock.
python tools/fetch_sidecar.py --list
python tools/fetch_sidecar.py llama_cpp cpu-x64
python tools/fetch_sidecar.py llama_cpp vulkan-x64

# 3. Fetch the pinned embedding runtime
./tools/fetch-sidecar.ps1 -Component onnxruntime -Variant win-x64

# 4. Full test suite, including tests that need a real desktop
cargo test --workspace -- --include-ignored

# 5. Verify the dependency boundary and the evaluation fixtures
python tools/check_network_deps.py --locked
python evals/validate.py

# 6. Build the desktop binary
cargo build -p tessara --features custom-protocol

# 7. Inspect what hardware would be selected on this machine
cargo run --release -p tsr-infer --bin tsr-hw -- \
    --model <path>/Qwen3VL-4B-Instruct-Q4_K_M.gguf \
    --mmproj <path>/mmproj-Qwen3VL-4B-Instruct-F16.gguf
```

Model files are not bundled and are never fetched automatically. They are downloaded only through an explicit approval step and are accepted only when their SHA-256 digest matches the pinned manifest in `tools/models.lock`.

[TO BE CONFIRMED] PLACEHOLDER: The repository's licence is recorded as "to be determined". A licence should be chosen and stated before the thesis is submitted, since the repository is referenced as project evidence.

[TO BE CONFIRMED] PLACEHOLDER: The repository folder and documentation use the name *Tessera*, while the compiled binary, the data directory (`%LOCALAPPDATA%\Tessara`) and the Cargo package are named `tessara`. The thesis uses *Tessera* throughout, and quotes the internal identifier exactly where a path or package name is given. Please confirm which spelling is intended, and whether the internal identifier should be renamed before submission.

APPENDIX C — CONFIGURATION REFERENCE

Table C.1 lists every configuration key, its type, its default and its effect.

Table C.1: Configuration reference (%LOCALAPPDATA%\Tessara\config.json)

Key	Type	Default	Effect
inference.backend	string	auto	Restricts inference to one backend: auto, cuda, rocm, vulkan, sycl, openvino, metal, opencl or cpu.
inference.device	string	(empty)	Restricts inference to devices matching an identifier, an index or part of a name.
inference.gpu_layers	integer	-1	Number of layers to offload. -1 offloads all; 0 keeps everything on the CPU.
inference.allow_fallback	boolean	true	When false, the application refuses to start inference rather than falling back from a forced choice.
inference.idle_unload_min	integer	10	Minutes of inactivity after which the worker unloads the model. 0 disables unloading.
capture.cadence_active_s	integer	5	Seconds between checks while the user is active (1 to 60).
capture.quality_threshold	float	0.55	Optical-character-recognition quality below which the vision rung is considered.
storage.retention_days	integer	90	Age at which stored items are pruned (1 to 365).

APPENDIX D — SELECTED TEST CASES

The following cases are representative of the test suite and are each cited in Chapter 6.

D.1 — A read never blocks the capture loop. *Purpose:* confirm that an accessibility read against an unresponsive application cannot stall capture. *Method:* read sixteen windows across nine applications with the production deadline worker, recording both the time the caller waited and the time the underlying read took. *Expected:* caller wait no greater than 300 ms. *Observed:* maximum 294 ms; abandoned reads continued for up to 732 ms in the background. *Interpretation:* the deadline, not the application, bounds the caller.

D.2 — A wrong key writes nothing to standard error. *Purpose:* confirm that the encryption library's own diagnostics cannot leak database content. *Method:* open the database with an incorrect key in a child process and capture its standard error stream. *Expected:* empty. *Observed:* empty. *Interpretation:* cipher logging is disabled as intended.

D.3 — Generic words do not over-block. *Purpose:* confirm that the blocklist does not suppress ordinary windows. *Method:* evaluate non-browser windows whose titles contain "login", "sign in" and "password". *Expected:* captured. *Observed:* captured. *Interpretation:* the title rule is scoped to browsers, so a developer's editor remains visible to the system.

D.4 — A fact cannot outlive its evidence. *Purpose:* confirm the provenance guarantee at the storage layer. *Method:* delete the last evidence row of a fact. *Expected:* the fact row disappears. *Observed:* deleted by trigger. *Interpretation:* provenance is enforced by the schema rather than by application code.

D.5 — No code path closes a commitment automatically. *Purpose:* confirm that nothing is marked done without the user. *Method:* a source scan that fails if any path other than the approval-token command writes a closed status, together with behavioural tests covering completion evidence, stated plans and other people's completions. *Expected:* no such path. *Observed:* none. *Interpretation:* automatic closure is structurally prevented, not merely avoided.

D.6 — Insertion refuses a changed or protected target. *Purpose:* confirm draft insertion safety. *Method:* attempt insertion into a password field, into a different

focused element, and into an element whose password state cannot be read. *Expected:* all three refused. *Observed:* all three refused. *Interpretation:* the unknown case is treated as the dangerous one.

D.7 — The interface is absent from screen captures. *Purpose:* confirm display-affinity exclusion. *Method:* place a magenta backdrop beneath the window and capture the monitor through Windows Graphics Capture and through GDI block transfer, with an unexcluded control window and an excluded control window in the same frames. *Expected:* the interface absent, the unexcluded control present, the excluded control absent. *Observed:* exactly that, for both the compact and expanded states. *Interpretation:* the mechanism works at the operating-system level; the named conferencing applications remain untested.

D.8 — The backend probe ignores an inherited environment. *Purpose:* confirm that the inference worker cannot be reconfigured by the parent's environment. *Method:* set a device-visibility variable in the parent and run the probe. *Expected:* the variable has no effect. *Observed:* no effect; a control run with the variable passed deliberately showed the opposite. *Interpretation:* environment stripping works.

APPENDIX E — GLOSSARY

Backend. A compiled build of the inference engine that targets one kind of acceleration hardware, such as CUDA or Vulkan. In Tessera a backend is an external artefact, not a linked library.

BM25. A ranking function for keyword search that rewards rare query terms and penalises unusually long documents.

Content-free logging. A logging discipline in which only allowlisted message strings and field names are emitted, so that user content cannot reach a log file even by mistake.

Dice coefficient. A similarity measure between two sets, twice the size of their intersection divided by the sum of their sizes. Used here for fuzzy quotation matching and for deduplication.

Display affinity. A Windows window attribute that excludes a window from screen capture and screen sharing.

GGUF. The container format used by llama.cpp for model weights; its header describes the architecture, layer count and quantisation of each tensor.

Indirect prompt injection. An attack in which adversarial instructions are placed in content that an application retrieves and feeds to a language model, rather than typed by the user.

Job Object. A Windows mechanism that groups processes so that limits apply collectively; used here to guarantee that the inference worker dies with the application.

Quantisation. Reducing the number of bits used to represent each model weight, trading accuracy for size and speed.

Reciprocal Rank Fusion. A method of combining several ranked lists using only the positions in each list rather than their scores.

Rung. One stage of Tessera's three-stage text acquisition ladder.

Sidecar. A separate process that performs work on behalf of an application, isolated from it so that its failures are contained.

SimHash. A fingerprint whose Hamming distance between two documents is small when the documents are similar; used for near-duplicate detection.

UI Automation. The Windows accessibility framework, which exposes other applications' interface elements and their text.

Vision-language model. A model that accepts images alongside text.

APPENDIX F — LIST OF ABBREVIATIONS

ADR Architecture Decision Record · **AES** Advanced Encryption Standard · **API** Application Programming Interface · **AVX** Advanced Vector Extensions · **BM25** Best Matching 25 · **CPU** Central Processing Unit · **CUDA** Compute Unified Device Architecture · **DPAPI** Data Protection Application Programming Interface · **DXGI** DirectX Graphics Infrastructure · **FTS** Full-Text Search · **GDI** Graphics Device Interface · **GGUF** GPT-Generated Unified Format · **GPU** Graphics Processing Unit · **HIP** Heterogeneous-Compute Interface for Portability · **HTTP** Hypertext Transfer Protocol · **IFSC** Indian Financial System Code · **IPC** Inter-Process Communication · **JSON** JavaScript Object Notation · **MVP** Minimum Viable Product · **NPU** Neural Processing Unit · **OCR** Optical Character Recognition · **ONNX** Open Neural Network Exchange · **OTP** One-Time Password · **OWASP** Open Worldwide Application Security Project · **PAN** Permanent Account Number · **PII** Personally Identifiable Information · **RAG** Retrieval-Augmented Generation · **RRF** Reciprocal Rank Fusion · **SBOM** Software Bill of Materials · **SQL** Structured Query Language · **SYCL** (a Khronos heterogeneous programming standard) · **TCP** Transmission Control Protocol · **TLS** Transport Layer Security · **UIA** UI Automation · **UPI** Unified Payments Interface · **VID** Virtual Identifier · **VLM** Vision-Language Model · **VRAM** Video Random-Access Memory · **WAL** Write-Ahead Logging · **WGC** Windows Graphics Capture · **WinRT** Windows Runtime

REFERENCES

[1] ggml-org, "llama.cpp: LLM inference in C/C++," GitHub repository. [Online]. Available: https://github.com/ggml-org/llama.cpp. [Accessed: 28-Oct-2026]. (Build b11900 is the pinned version used in this work.)

[2] Qwen Team, "Qwen3-VL Technical Report," arXiv:2511.21631 [cs.CV], 2025. [Online]. Available: https://arxiv.org/abs/2511.21631

[3] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[4] SQLite Consortium, "SQLite FTS5 Extension." [Online]. Available: https://www.sqlite.org/fts5.html. [Accessed: 28-Oct-2026].

[5] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder and F. Wei, "Multilingual E5 Text Embeddings: A Technical Report," arXiv:2402.05672 [cs.CL], 2024. [Online]. Available: https://arxiv.org/abs/2402.05672

[6] G. V. Cormack, C. L. A. Clarke and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in Proc. 32nd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval, Boston, MA, 2009, pp. 758–759.

[7] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), 2023. arXiv:2302.12173. [Online]. Available: https://arxiv.org/abs/2302.12173

[8] M. S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," in Proc. 34th Annual ACM Symposium on Theory of Computing (STOC '02), Montreal, 2002, pp. 380–388.

[9] G. S. Manku, A. Jain and A. Das Sarma, "Detecting Near-Duplicates for Web Crawling," in Proc. 16th Int. Conf. on World Wide Web (WWW '07), Banff, 2007, pp. 141–150.

[10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020. arXiv:2005.11401.

[11] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto and P. Fung, "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.

[12] L. R. Dice, "Measures of the Amount of Ecologic Association Between Species," Ecology, vol. 26, no. 3, pp. 297–302, 1945.

[13] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," arXiv:2211.09527 [cs.CL], 2022. [Online]. Available: https://arxiv.org/abs/2211.09527

[14] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications." [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/. [Accessed: 28-Oct-2026].

[15] Zetetic LLC, "SQLCipher: Full Database Encryption for SQLite — Design and Documentation." [Online]. Available: https://www.zetetic.net/sqlcipher/design/. [Accessed: 28-Oct-2026].

[16] Microsoft Corporation, "Manage Recall for Windows clients," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/windows/client-management/manage-recall. [Accessed: 28-Oct-2026].

[17] International Organization for Standardization, ISO/IEC 7812-1:2017, "Identification cards — Identification of issuers — Part 1: Numbering system," Geneva, 2017. (Annex B specifies the Luhn check-digit formula.)

[18] J. Verhoeff, "Error Detecting Decimal Codes," Mathematical Centre Tract 29, Mathematisch Centrum, Amsterdam, 1969.

[19] Tauri Programme within The Commons Conservancy, "Tauri 2 Documentation." [Online]. Available: https://v2.tauri.app/. [Accessed: 28-Oct-2026].

[20] Microsoft Corporation, "UI Automation Overview," Windows Developer Documentation. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/framework/ui-automation/ui-automation-overview. [Accessed: 28-Oct-2026].

[21] Microsoft Corporation, "Screen capture with Windows.Graphics.Capture," Windows Developer Documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/uwp/audio-video-camera/screen-capture. [Accessed: 28-Oct-2026].

[22] Microsoft Corporation, "Job Objects," Win32 Developer Documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/procthread/job-objects. [Accessed: 28-Oct-2026].

[23] Microsoft Corporation, "SetWindowDisplayAffinity function," Win32 Developer Documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-setwindowdisplayaffinity. [Accessed: 28-Oct-2026].

[24] Microsoft Corporation, "How to: Use Data Protection (DPAPI)," .NET Documentation. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/standard/security/how-to-use-data-protection. [Accessed: 28-Oct-2026].

[25] ONNX Runtime Developers, "ONNX Runtime." [Online]. Available: https://onnxruntime.ai/. [Accessed: 28-Oct-2026].

[26] R. Jung, J.-H. Jourdan, R. Krebbers and D. Dreyer, "Safe Systems Programming in Rust," Communications of the ACM, vol. 64, no. 4, pp. 144–152, 2021.

[27] Unique Identification Authority of India, "Aadhaar Numbering Scheme" (Verhoeff check-digit specification for the twelve-digit Aadhaar number). [Online]. Available: https://uidai.gov.in/. [Accessed: 28-Oct-2026].

[28] Microsoft Corporation, "Microsoft Edge WebView2 Documentation." [Online]. Available: https://learn.microsoft.com/en-us/microsoft-edge/webview2/. [Accessed: 28-Oct-2026].